

SOLIDWORKS API (.NET VBA C#) — Examples (4/50): Add Menu and Menu Item Example VB.NET- Analyze_Text_and_Geometry_in_Glot_Flat_S

SOLIDWORKS API Documentation (converted from CHM)

Add Menu and Menu Item Example (VB.NET)

This example shows how to add a menu and menu item to a part.

```

-----
Preconditions:
1. Verify that the SOLIDWORKS API SDK add-in templates for your version
  of Microsoft Visual Studio are installed.
2. In Microsoft Visual Studio, create a Visual Basic project using the
  SwVBAddin template.
3. Name the project AddMenuItemVBNET.
4. Copy and paste SwAddin into SwAddin.vb of your project.
5. Click Project > AddMenuItemVBNET Properties > Debug.
6. Select Start external program and type the pathname of your
  SOLIDWORKS executable.
7. Replace the images array elements with the pathnames of
  your image files.
8. Press F5 to start debugging this add-in.

Postconditions:
1. In SOLIDWORKS, click File > New > Part > OK.
2. Click Tools > MyMenu. Note the size of
  the MyMenuItem button.
3. Click MyMenuItem and click OK to close the message box.
4. Exit SOLIDWORKS.
5. Change the size of the text and other items on your computer.
  For example, in Windows 7:
  a. Click Start > Control Panel > Appearance and
    Personalization > Display.
  b. Select a different size.
  c. Click Apply.
  d. Click Log off now.
  e. Log back in.
6. Start SOLIDWORKS.
7. Repeat steps 1 and 2.
-----
'SwAddin
Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SolidWorksTools
Imports SolidWorksTools.File
Imports System.Windows.Forms
Imports System.Collections.Generic
Imports System.Diagnostics

<Guid("2b89b82f-2984-439a-a818-3c79180e7916")> _
<ComVisible(True)> _
<SwAddin(
  Description:="AddMenuItemVBNET description", _
  Title:="AddMenuItemVBNET", _
  LoadAtStartup:=True _
)>

Public Class SwAddin
  Implements SolidWorks.Interop.swpublished.SwAddin

#Region "Local Variables"
  Dim WithEvents iSwApp As SldWorks
  Dim addinID As Integer
  Dim openDocs As Hashtable
  Dim SwEventPtr As SldWorks

  ' Public properties
  ReadOnly Property SwApp() As SldWorks
    Get
      Return iSwApp
    End Get
  End Property

  ReadOnly Property OpenDocumentsTable() As Hashtable
    Get
      Return openDocs
    End Get
  End Property
#End Region

#Region "SolidWorks Registration"
  <ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)
    ' Get Custom Attribute: SwAddinAttribute
    Dim attributes() As Object
    Dim SwAttr As SwAddinAttribute = Nothing

    attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

    If attributes.Length > 0 Then
      SwAttr = DirectCast(attributes(0), SwAddinAttribute)
    End If
    Try
      Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
      Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

      Dim keyname As String = "SOFTWARE\SolidWorks\Addins\{" + t.GUID.ToString() + "}"
      Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)
      addinkey.SetValue(Nothing, 0)
      addinkey.SetValue("Description", SwAttr.Description)
      addinkey.SetValue("Title", SwAttr.Title)

      keyname = "Software\SolidWorks\AddinStartup\{" + t.GUID.ToString() + "}"
      addinkey = hkcu.CreateSubKey(keyname)
      addinkey.SetValue(Nothing, SwAttr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
    Catch nl As System.NullReferenceException
      Console.WriteLine("There was a problem registering this dll: SwAttr is null.\n " & nl.Message)
      System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: SwAttr is null.\n" & nl.Message)
    Catch e As System.Exception
  
```

```

        Console.WriteLine("There was a problem registering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: " & e.Message)
    End Try
End Sub

<ComUnregisterFunction()> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Try
        Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SolidWorks\Addins\{" + t.GUID.ToString() + "}"
        hklm.DeleteSubKey(keyname)

        keyname = "Software\SolidWorks\AddInsStartup\{" + t.GUID.ToString() + "}"
        hkcu.DeleteSubKey(keyname)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem unregistering this dll: Swattr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: Swattr is null.\n" & nl.Message)
    Catch e As System.Exception
        Console.WriteLine("There was a problem unregistering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: " & e.Message)
    End Try
End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSWApp = ThisSW
    addinID = Cookie

    ' Set up callbacks
    iSWApp.SetAddinCallbackInfo2(0, Me, addinID)

    ' Set up the menu item
    AddMenuItem()

    ' Set up the Event Handlers
    SwEventPtr = iSWApp
    openDocs = New Hashtable
    AttachEventHandlers()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW
    DetachEventHandlers()

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSWApp)
    iSWApp = Nothing
    'The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect()
    GC.WaitForPendingFinalizers()

    GC.Collect()
    GC.WaitForPendingFinalizers()

    DisconnectFromSW = True
End Function
#End Region

#Region "UI Methods"
' Add menu and menu item
Public Sub AddMenuItem()
    Dim thisAssembly1 As Assembly
    Dim images As String() = New String(2) {}
    Dim menuId As Integer = 0

    thisAssembly1 = System.Reflection.Assembly.GetAssembly(Me.GetType())

    images(0) = "Pathname_to_menu_item_nxn_image"
    images(1) = "Pathname_to_menu_item_nnnxn_image"
    images(2) = "Pathname_to_menu_item_nnnxn_image"

    menuId = SwApp.AddMenu(swDocumentTypes_e.swDocPART, "MyMenu", 0)
    menuId = iSWApp.AddMenuItem5(swDocumentTypes_e.swDocPART, addinID, "MenuItem@MyMenu", 0, "MyMenuCallback", "MyMenuEnableMethod", "My menu item",
images)
End Sub

Function CompareIDs(ByVal storedIDs() As Integer, ByVal addinIDs() As Integer) As Boolean

    Dim storeList As New List(Of Integer)(storedIDs)
    Dim addinList As New List(Of Integer)(addinIDs)

    addinList.Sort()
    storeList.Sort()

    If Not addinList.Count = storeList.Count Then

        Return False
    Else

        For i As Integer = 0 To addinList.Count - 1
            If Not addinList(i) = storeList(i) Then

                Return False
            End If
        Next
    End If

    Return True
End Function
#End Region

#Region "UI callbacks"
Public Sub MyMenuCallback()
    MessageBox.Show("Callback function called.")
End Sub
#End Region

#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()

    'Listen for events on all currently open documents

```

```

        AttachEventsToAllDocuments()
    End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    'Close events on all currently open documents
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        'Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            'Remove the pair from the hash
            docHandler.DetachEventHandlers()
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    Try
        AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub DetachSWEvents()
    Try
        RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        Else
            Dim docHandler As DocumentEventHandler = openDocs(modDoc)
            If Not docHandler Is Nothing Then
                docHandler.ConnectModelViews()
            End If
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelDocEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function
#End Region

```

End Class

Add Multibody Part Explode Step Example (VBA)

This example shows how to create an explode view of a multibody part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\multibody\multi_Inter.sldprt.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Creates Exploded View1 in ConfigurationManager.  
' 2. Adds Chain1 to Exploded View1.  
' 3. Displays the Exploded View1 in its exploded state.  
' 4. Inspect the Immediate window, the ConfigurationManager,  
'    and the graphics area.  
'  
' Note: Because the model is used elsewhere, do not save any changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.PartDoc  
Dim config As SldWorks.Configuration  
Dim configName As String  
Dim swMdl As SldWorks.ModelDoc2  
Dim explStep As SldWorks.PartExplodeStep  
Dim comp As SldWorks.Body2  
Dim var As Variant  
Dim varViews As Variant  
Dim configVar As Variant  
Dim boolstatus As Boolean  
Dim i As Long  
Dim errCode As Long
```

Sub main()

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swMdl = swModel
```

```
configVar = swMdl.GetConfigurationNames
```

```
Call swModel.CreateExplodedView  
varViews = swModel.GetExplodedViewNames(configVar(0))  
configName = swModel.GetExplodedViewConfigurationName(varViews(0))  
Set config = swMdl.GetConfigurationByName(configName)
```

```
swModel.ShowExploded True, varViews(0)
```

```
'Select the bodies to move
```

```
boolstatus = swModel.Extension.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 1, Nothing, 0)
```

```
'Add an explode step without selecting an explode direction entity
```

```
'Defaults to the Z-direction manipulator index
```

```
Set explStep = config.AddPartExplodeStep("Exploded View1", 0.07, -1, False, True, errCode)
```

```
Call swMdl.EditRebuild3
```

```
'Get bodies moved in the explode step
```

```
var = explStep.GetBodies()
```

```
Debug.Print "Explode view: " & varViews(0)
```

```
Debug.Print "Explode step: " & explStep.Name
```

```
Debug.Print "Explode distance (m): " & explStep.ExplodeDistance
```

```
Debug.Print "Reverse translation direction? " & explStep.ReverseTranslationDirection
```

```
Debug.Print "Automatically space components on drag? " & explStep.AutoSpaceBodiesOnDrag
```

```
Debug.Print "Bodies moved in the explode step:"
```

```
For i = 0 To UBound(var)
```

```
Set comp = var(i)
```

```
Debug.Print " " & comp.Name
```

```
Next
```

```
End Sub
```

Add Multibody Part Explode Step Example (C#)

This example shows how to create an explode view of a multibody part.

```
// -----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\multibody\multi_Inter.sldprt.  
// 2. Open an Immediate window.  
//  
// Postconditions:  
// 1. Creates Exploded View1 in ConfigurationManager.  
// 2. Adds Chain1 to Exploded View1.  
// 3. Displays the Exploded View1 in its exploded state.  
// 4. Inspect the Immediate window, the ConfigurationManager,  
// and the graphics area.  
//  
// Note: Because the model is used elsewhere, do not save any changes.  
// -----
```

```
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System.Diagnostics;
```

```
namespace MultibodyPartExplode_CSharp  
{
```

```
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            PartDoc swModel;  
  
            Configuration config;  
  
            string configName;  
  
            ModelDoc2 swMdl;  
  
            PartExplodeStep explStep;  
  
            Body2 comp;  
  
            object[] var;  
  
            object[] varViews;  
  
            object[] configVar;  
  
            bool boolstatus;  
  
            int i;  
  
            int errCode;  
  
            swModel = (PartDoc)swApp.ActiveDoc;  
  
            swMdl = (ModelDoc2)swModel;
```

```

configVar = (object[])swMdl.GetConfigurationNames();

swModel.CreateExplodedView();

varViews = (object[])swModel.GetExplodedViewNames((string)configVar[0]);

configName = swModel.GetExplodedViewConfigurationName((string)varViews[0]);

config = (Configuration)swMdl.GetConfigurationByName(configName);

swModel.ShowExploded(true, (string)varViews[0]);

// Select the bodies to move

boolstatus = swMdl.Extension.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, true, 1, null, 0);

// Add an explode step without selecting an explode direction entity

// Defaults to the Z-direction manipulator index

explStep = (PartExplodeStep)config.AddPartExplodeStep("Exploded View1", 0.07, -1, false, true, out errCode);

swMdl.EditRebuild3();

// Get bodies moved in the explode step

var = (object[])explStep.GetBodies();

Debug.Print("Explode view: " + varViews[0]);

Debug.Print("Explode step: " + explStep.Name);

Debug.Print("Explode distance (m): " + explStep.ExplodeDistance);

Debug.Print("Reverse translation direction? " + explStep.ReverseTranslationDirection);

Debug.Print("Automatically space components on drag? " + explStep.AutoSpaceBodiesOnDrag);

Debug.Print("Bodies moved in the explode step:");

for (i = 0; i <= var.GetUpperBound(0); i++)
{
    comp = (Body2)var[i];

    Debug.Print(" " + comp.Name);
}
}

```

```
/// <summary>
///     ''' The SldWorks swApp variable is pre-assigned for you.
///     ''' </summary>
public SldWorks swApp;

}
}
```

Add Multibody Part Explode Step Example (VB.NET)

This example shows how to create an explode view of a multibody part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\multibody\multi_Inter.sldprt.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Creates Exploded View1 in ConfigurationManager.  
' 2. Adds Chain1 to Exploded View1.  
' 3. Displays the Exploded View1 in its exploded state.  
' 4. Inspect the Immediate window, the ConfigurationManager,  
'    and the graphics area.  
'  
' Note: Because the model is used elsewhere, do not save any changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()  
  
        Dim swModel As PartDoc  
  
        Dim config As Configuration  
  
        Dim configName As String  
  
        Dim swMdl As ModelDoc2  
  
        Dim explStep As PartExplodeStep  
  
        Dim comp As Body2  
  
        Dim var As Object  
  
        Dim varViews As Object  
  
        Dim configVar As Object  
  
        Dim boolstatus As Boolean  
  
        Dim i As Long  
  
        Dim errCode As Long  
  
        swModel = swApp.ActiveDoc  
  
        swMdl = swModel  
  
        configVar = swMdl.GetConfigurationNames  
  
        Call swModel.CreateExplodedView
```

```

varViews = swModel.GetExplodedViewNames(configVar(0))

configName = swModel.GetExplodedViewConfigurationName(varViews(0))

config = swMdl.GetConfigurationByName(configName)

swModel.ShowExploded(True, varViews(0))

'Select the bodies to move

boolstatus = swMdl.Extension.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 1, Nothing, 0)

'Add an explode step without selecting an explode direction entity

'Defaults to the Z-direction manipulator index

explStep = config.AddPartExplodeStep("Exploded View1", 0.07, -1, False, True, errCode)

Call swMdl.EditRebuild3

'Get bodies moved in the explode step

var = explStep.GetBodies()

Debug.Print("Explode view: " & varViews(0))

Debug.Print("Explode step: " & explStep.Name)

Debug.Print("Explode distance (m): " & explStep.ExplodeDistance)

Debug.Print("Reverse translation direction? " & explStep.ReverseTranslationDirection)

Debug.Print("Automatically space components on drag? " & explStep.AutoSpaceBodiesOnDrag)

Debug.Print("Bodies moved in the explode step:")

For i = 0 To UBound(var)

    comp = var(i)

    Debug.Print(" " & comp.Name)

Next

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Add Objects to Selection List Example (C#)

This example shows how to populate the selection list with objects that have not been pre-selected through the user interface.

```
// -----
// Preconditions:
// 1. Open Public_documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\multi_local.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Suspends the selection list containing Sketch2.
// 2. Populates another selection list with six sketch objects.
// 3. Gets the type of the last object in the selection list.
// 4. Appends the last selection list of six objects to the previously suspended
//    selection list that contains Sketch2, placing Sketch2 at the top of the
//    combined and resumed selection list.
// 5. Places Sketch2 in edit mode.
// 6. Inspect the Immediate window and graphics area.
// 7. Discard changes to the model.
//
// NOTE: Because the model is used elsewhere, do not save any changes.
// -----

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace ResumeSelectionList_CSharp
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            SelectionMgr selMgr;
            SelectData selData;
            FeatureManager featMgr;
            Feature feat;
            Dispatcher[] arrObjIn = new Dispatcher[6];
            Feature[] selObjs = new Feature[6];
            object selObj;
            int featCount;
            int i;
            string typeName;
            int j;
            int numAdded;
            bool boolstatus;
            int ret;
        }
    }
}
```

```

int count;

swModel = (ModelDoc2)swApp.ActiveDoc;
selMgr = (SelectionMgr)swModel.SelectionManager;
selData = selMgr.CreateSelectData();
featMgr = swModel.FeatureManager;

featCount = featMgr.GetFeatureCount(true);
feat = (Feature)swModel.FirstFeature();
j = 0;
for (i = 0; i <= featCount; i++)
{
    if ((feat != null))

    {

        typeName = feat.Name;
        typeName = typeName.ToUpper();

        if ("SKE" == typeName.Substring(0, 3))
        {
            selObjs[j] = feat;
            arrObjIn[j] = new DispatcherWrapper(selObjs[j]);

            j = j + 1;
        }

        feat = (Feature)feat.GetNextFeature();
    }
}

// Add one object to the current selection list
boolstatus = swModel.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 0, null, 0);

// Start a new selection list
ret = selMgr.SuspendSelectionList();
Debug.Print("The current selection list with " + ret + " object (Sketch2) is suspended.");

// Add six objects to the new selection list
numAdded = selMgr.AddSelectionListObjects((selObjs), selData);
Debug.Print("A new selection list is started.");

// Get number of objects in the new selection list (should be 6)
count = selMgr.GetSelectedObjectCount();
Debug.Print("The selection list now contains " + count + " objects.");

// Get the last object in the new selection list
selObj = selMgr.GetSelectedObject6(count, -1);
Debug.Print("The last object in the selection list is of swSelectType_e = " + selMgr.GetSelectedObjectType3(count, -1)
+ ".");

// Resume the previously suspended selection list (Sketch2), appending the new selection list to it
selMgr.ResumeSelectionList2(true);
Debug.Print("The previous selection list is resumed and appended with the new selection list.");

// Get the number of objects in the selection list (should be 7)

```

```
        count = selMgr.GetSelectedObjectCount();
        Debug.Print("The selection list now contains " + count + " objects, including Sketch2, which is at the top of the
selection list.");

        // Edit sketch2
        swModel.EditSketch();
    }
    /// <summary>
    ///     ''' The SldWorks swApp variable is pre-assigned for you.
    ///     ''' </summary>
    public SldWorks swApp;
}
}
```

Add Objects to Selection List Example (VBA)

This example shows how to populate the selection list with objects that have not been pre-selected through the user interface.

```
'-----
' Preconditions:
' 1. Open Public_documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\multi_local.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Suspends the selection list containing Sketch2.
' 2. Populates another selection list with six sketch objects.
' 3. Gets the type of the last object in the selection list.
' 4. Appends the last selection list of six objects to the previously suspended
'    selection list that contains Sketch2, placing Sketch2 at the top of the
'    combined and resumed selection list.
' 5. Places Sketch2 in edit mode.
' 6. Inspect the Immediate window and graphics area.
' 7. Discard changes to the model.
'
' NOTE: Because the model is used elsewhere, do not save any changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim selMgr As SldWorks.SelectionMgr
Dim selData As SldWorks.SelectData
Dim featMgr As SldWorks.FeatureManager
Dim feat As SldWorks.Feature
Dim selObj() As Object
Dim selObj As Object
Dim featCount As Long
Dim i As Integer
Dim typeName As String
Dim j As Integer
Dim numAdded As Long
Dim boolstatus As Boolean
Dim ret As Long
Dim count As Long
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set selMgr = swModel.SelectionManager
Set selData = selMgr.CreateSelectData
Set featMgr = swModel.FeatureManager
```

```
featCount = featMgr.GetFeatureCount(True)
Set feat = swModel.FirstFeature
j = 0
ReDim selObj(j + 1)
For i = 0 To featCount
    If Not feat Is Nothing Then
```

```
        typeName = feat.Name
        typeName = UCase(typeName)
        If "SKETCH" = Left(typeName, 6) Then
            Set selObj(j) = feat
            j = j + 1
            ReDim Preserve selObj(j + 1)
        End If
        Set feat = feat.GetNextFeature
    End If
```

```
End If
Next i
```

```
' Add one object to the current selection list
boolstatus = swModel.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
' Start a new selection list
ret = selMgr.SuspendSelectionList
Debug.Print "The current selection list with " & ret & " object (Sketch2) is suspended."
```

```
' Add six objects to the new selection list
numAdded = selMgr.AddSelectionListObjects((selObj), selData)
Debug.Print "A new selection list is started."
```

```
' Get number of objects in the new selection list (should be 6)
count = selMgr.GetSelectedObjectCount
Debug.Print "The selection list now contains " & count & " objects."
```

```
' Get the last object in the new selection list
Set selObj = selMgr.GetSelectedObject6(count, -1)
Debug.Print "The last object in the selection list is of swSelectType_e = " & selMgr.GetSelectedObjectType3(count, -1) & "."
```

```
' Resume the previously suspended selection list (Sketch2), appending the new selection list to it
selMgr.ResumeSelectionList2 (True)
Debug.Print "The previous selection list is resumed and appended with the new selection list."

' Get the number of objects in the selection list (should be 7)
count = selMgr.GetSelectedObjectCount
Debug.Print "The selection list now contains " & count & " objects, including Sketch2, which is at the top of the selection list."

' Edit sketch2
swModel.EditSketch
```

End Sub

Add Objects to Selection List Example (VB.NET)

This example shows how to populate the selection list with objects that have not been pre-selected through the user interface.

```
' -----
' Preconditions:
' 1. Open Public_documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\multi_local.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Suspends the selection list containing Sketch2.
' 2. Populates another selection list with six sketch objects.
' 3. Gets the type of the last object in the selection list.
' 4. Appends the last selection list of six objects to the previously suspended
'    selection list that contains Sketch2, placing Sketch2 at the top of the
'    combined and resumed selection list.
' 5. Places Sketch2 in edit mode.
' 6. Inspect the Immediate window and graphics area.
' 7. Discard changes to the model.
'
' NOTE: Because the model is used elsewhere, do not save any changes.
' -----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim selMgr As SelectionMgr
        Dim selData As SelectData
        Dim featMgr As FeatureManager
        Dim feat As Feature
        Dim selObjs() As Object
        Dim selObj As Object
        Dim featCount As Integer
        Dim i As Integer
        Dim typeName As String
        Dim j As Integer
        Dim numAdded As Integer
        Dim boolstatus As Boolean
        Dim ret As Integer
        Dim count As Integer

        swModel = swApp.ActiveDoc
        selMgr = swModel.SelectionManager
        selData = selMgr.CreateSelectData
        featMgr = swModel.FeatureManager

        featCount = featMgr.GetFeatureCount(True)
        feat = swModel.FirstFeature
        j = 0
        ReDim selObjs(j + 1)
        For i = 0 To featCount
            If Not feat Is Nothing Then
```

```

        typeName = feat.Name
        typeName = UCase(typeName)
        If "SKETCH" = Left(typeName, 6) Then
            selObjs(j) = feat
            j = j + 1
            ReDim Preserve selObjs(j + 1)
        End If
        feat = feat.GetNextFeature
    End If
Next i

' Add one object to the current selection list
boolstatus = swModel.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

' Start a new selection list
ret = selMgr.SuspendSelectionList
Debug.Print("The current selection list with " & ret & " object (Sketch2) is suspended.")

' Add six objects to the new selection list
numAdded = selMgr.AddSelectionListObjects((selObjs), selData)
Debug.Print("A new selection list is started.")

' Get number of objects in the new selection list (should be 6)
count = selMgr.GetSelectedObjectCount
Debug.Print("The selection list now contains " & count & " objects.")

' Get the last object in the new selection list
selObj = selMgr.GetSelectedObject6(count, -1)
Debug.Print("The last object in the selection list is of swSelectType_e = " & selMgr.GetSelectedObjectType3(count, -1) &
".")

' Resume the previously suspended selection list (Sketch2), appending the new selection list to it
selMgr.ResumeSelectionList2(True)
Debug.Print("The previous selection list is resumed and appended with the new selection list.")

' Get the number of objects in the selection list (should be 7)
count = selMgr.GetSelectedObjectCount
Debug.Print("The selection list now contains " & count & " objects, including Sketch2, which is at the top of the selection
list.")

' Edit sketch2
swModel.EditSketch()

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Add or Remove Curvature Combs Example (VBA)

This example shows how to add or remove curvature combs to a curved sketch segment.

.....

' Preconditions: Curved sketch segment is selected.

,

' Postconditions: Curvature combs are added or removed.

.....

Option Explicit

Sub main()

Dim swApp As SldWorks.SldWorks

Dim swModel As SldWorks.ModelDoc2

Dim nResponse As Integer

Dim bRet As Boolean

Set swApp = Application.SldWorks

Set swModel = swApp.ActiveDoc

nResponse = MsgBox("Add curvature combs?", vbYesNo)

If nResponse = vbYes Then

swModel.**InspectCurvature**

Else

swModel.**RemoveInspectCurvature**

End If

End Sub

Add Ordinate Dimension Example (VBA)

This example shows how to add an ordinate dimension.

```
.....
' Preconditions:
'
' 1) Drawing document is open.
' 2) Two horizontal parallel edges are selected.
'
' Postconditions: Ordinate dimension added.
.....

Option Explicit

Public Enum swAddOrdinateDims_e
    swOrdinate = 1
    swVerticalOrdinate = 2
    swHorizontalOrdinate = 3
End Enum

Public Enum swCreateOrdDimError_e
    swCreateOrdDimErr_Undefined = -1      ' If encountered, report as SPR.
    swCreateOrdDimErr_Success = 0
    swCreateOrdDimErr_GenFailure = 1      ' MSG_SDIM_BAD_DIM
    swCreateOrdDimErr_GenNoInternalDims = 2  ' MSG_NO_INTERNAL_DIMS_IN_DSKETCH
    swCreateOrdDimErr_GenBadSel = 3        ' MSG_DIM_REF_NOCREATE
    swCreateOrdDimErr_GenNeedModelLoaded = 4  ' MSG_CANNOT_DIM_GHOST_IN3D
    swCreateOrdDimErr_GenSamePartOnly = 5    ' MSG_DIM_EDIT_PART
    swCreateOrdDimErr_GenExtraSelection = 6   ' MSG_DIM_TOO_MANY_SELECT
    swCreateOrdDimErr_OrdFailure = 7         ' MSG_BAD_ORDINATE_DIM0
    swCreateOrdDimErr_OrdDupInGroup = 8      ' MSG_BAD_ORDINATE_DIM1
    swCreateOrdDimErr_OrdBadDir = 9         ' MSG_BAD_ORDINATE_DIM2
End Enum

Sub main()
    Dim swApp      As SldWorks.SldWorks
    Dim swModel     As SldWorks.modelDoc
    Dim swDraw      As SldWorks.DrawingDoc
    Dim nRetVal     As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    ' Horizontal ordinate ignores X placement
    nRetVal = swDraw.AddOrdinateDimension2(swHorizontalOrdinate, -0.1, 0.02, 0#)

End Sub
```

Add Reference Dimension Example (C#)

This example shows how to add a reference dimension to a model in a drawing.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Activates a drawing view, selects an edge on the model, and
//    creates a dimension.
// 3. Prints to the Immediate window whether the dimension
//    is a reference dimension.
// 4. Examine the Immediate window.
//
// NOTE: Because the drawing document is used elsewhere, do not
// save any changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ISReferenceDimCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20.SLDDRW";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawingDoc = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            status = swDrawingDoc.ActivateView("Drawing View1");
            status = swModelDocExt.SelectByID2("", "EDGE", 0.512187343878665, 0.498697444621999, 249.953027873291, false, 0, null, 0);
            swDisplayDimension = (DisplayDimension)swModelDocExt.AddDimension(0.698326046410311, 0.699228314873418, 0, (int)swSmartDimensionDirection_e.swSmartDimensionDirection_Up);
            Debug.Print("Is reference dimension? " + swDisplayDimension.IsReferenceDim());

            swModel.ClearSelection2(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Add Reference Dimension Example (VBA)

This example shows how to add a reference dimension to a model in a drawing.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified drawing document.  
' 2. Activates a drawing view, selects an edge on the model, and  
'    creates a dimension.  
' 3. Prints to the Immediate window whether the dimension  
'    is a reference dimension.  
' 4. Examine the Immediate window.  
'  
' NOTE: Because the drawing document is used elsewhere, do not  
' save any changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawingDoc As SldWorks.DrawingDoc  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swDisplayDimension As SldWorks.DisplayDimension  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20.SLDDRW"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swDrawingDoc = swModel  
    Set swModelDocExt = swModel.Extension  
  
    status = swDrawingDoc.ActivateView("Drawing View1")  
    status = swModelDocExt.SelectByID("", "EDGE", 0.512187343878665, 0.498697444621999, 249.953027873291, False, 0, Nothing, 0)  
    Set swDisplayDimension = swModelDocExt.AddDimension(0.698326046410311, 0.699228314873418, 0, swSmartDimensionDirection_e.swSmartDimensionDirection_Up)  
    Debug.Print ("Is reference dimension? " & swDisplayDimension.IsReferenceDim)  
  
    swModel.ClearSelection2 True  
  
End Sub
```

Add Reference Dimension Example (VB.NET)

This example shows how to add a reference dimension to a model in a drawing.

```

-----
Preconditions:
1. Verify that the specified drawing document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified drawing document.
2. Activates a drawing view, selects an edge on the model, and
   creates a dimension.
3. Prints to the Immediate window whether the dimension
   is a reference dimension.
4. Examine the Immediate window.
NOTE: Because the drawing document is used elsewhere, do not
save any changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDrawingDoc As DrawingDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swDisplayDimension As DisplayDimension
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20.SLDDRW"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawingDoc = swModel
        swModelDocExt = swModel.Extension

        status = swDrawingDoc.ActivateView("Drawing View1")
        status = swModelDocExt.SelectByID("", "EDGE", 0.512187343878665, 0.498697444621999, 249.953027873291, False, 0, Nothing, 0)
        swDisplayDimension = swModelDocExt.AddDimension(0.698326046410311, 0.699228314873418, 0, swSmartDimensionDirection_e.swSmartDimensionDirection_Up)
        Debug.Print("Is reference dimension? " & swDisplayDimension.IsReferenceDim)

        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Add Row to Design Table Example (VBA)

This example shows how to add a row to a design table. This example also adds a configuration to the part document.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a design table.  
' 2. Select the design table on the ConfigurationManager  
'    tab (expand Tables and select Design Table).  
'  
' Postconditions: Adds a row and a configuration  
' named 190 to the part document.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDesignTable As SldWorks.DesignTable  
    Dim cells(1) As String  
    Dim boolstatus As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDesignTable = swModel.GetDesignTable  
  
    ' Name of configuration for newly created row  
    cells(0) = "190"  
  
    ' Data for Column B of newly created row  
    cells(1) = "S"  
  
    swDesignTable.EditTable  
    boolstatus = swDesignTable.AddRow((cells))  
    boolstatus = swDesignTable.UpdateTable(SwConst.swDesignTableUpdateOptions_e.swUpdateDesignTableAll, True)  
  
End Sub
```

Add Shortcut Menus to Add-ins Example (C#)

This example shows how to create a shortcut menu that launches from a third-party icon in the context-sensitive menus of part faces.

```
//-----
// Preconditions:
// 1. In Microsoft Visual Studio 2010, or later, create an SOLIDWORKS add-in
//    project using Visual C# > SwCSharpAddin.
// 2. Name the project SwCSharpAddin1.
// 3. Copy and paste this code into SwAddin.cs of your C# project.
// 4. Click Project > SwCSharpAddin1 Properties > Debug.
// 5. Select Start external program and type the path to sldworks.exe. Include
//    sldworks.exe in the path.
// 6. Press F5 to start debugging SOLIDWORKS Design with this add-in.
//
// Postconditions:
// 1. In SOLIDWORKS Design, click Tools > C# Add-in > CreateCube.
// 2. Click a face on the cube.
// 3. Click the third-party icon in the face's context-sensitive menu.
// 4. Click a shortcut menu item and inspect the Immediate Window.
// 5. Click Tools > Options > System Options > Colors > select a different
//    Background > OK.
// 6. Displays the new background. Click OK to close the message box.
// 7. Displays the FeatureManager design tree decimal color value. Click OK
//    to close the message box.
//-----

// SwAddin.cs:
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Windows.Forms;

namespace SwCSharpAddin1
{
    /// <summary>
    /// Create third-party pop-up menus.
    /// </summary>
    [Guid("144d6667-43ec-4733-ae26-867c9aa63fe6"), ComVisible(true)]
    [SwAddin(
        Description = "SwCSharpAddin1 description",
        Title = "SwCSharpAddin1",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        ISldWorks iSwApp;
        ICommandManager iCmdMgr;
        int addinID;
        int registerID;
        bool resultCode;
        IModelDoc2 model;
        IFrame frame;

        #region Event Handler Variables
        Hashtable openDocs;
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr;
        #endregion

        #region Property Manager Variables
        UserPMPPage ppage;
        #endregion

        // Public Properties
        public ISldWorks SwApp
        {
            get { return iSwApp; }
        }
        public ICommandManager CmdMgr
        {
            get { return iCmdMgr; }
        }

        public Hashtable OpenDocs
        {
            get { return openDocs; }
        }
    }
}
```

```

#endregion

#region SOLIDWORKS Registration
[ComRegisterFunctionAttribute]
public static void RegisterFunction(Type t)
{
    #region Get Custom Attribute: SwAddinAttribute
    SwAddinAttribute SwAttr = null;
    Type type = typeof(SwAddin);
    foreach (System.Attribute attr in type.GetCustomAttributes(false))
    {
        if (attr is SwAddinAttribute)
        {
            SwAttr = attr as SwAddinAttribute;
            break;
        }
    }
    #endregion

    Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
    Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

    string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "}";
    Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
    addinkey.SetValue(null, 0);

    addinkey.SetValue("Description", SwAttr.Description);
    addinkey.SetValue("Title", SwAttr.Title);

    keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "}";
    addinkey = hkcu.CreateSubKey(keyname);
    addinkey.SetValue(null, Convert.ToInt32(SwAttr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
}

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction(Type t)
{
    Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
    Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

    string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "}";
    hklm.DeleteSubKey(keyname);

    keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "}";
    hkcu.DeleteSubKey(keyname);
}

#endregion

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    //Setup callbacks
    iSwApp.SetAddinCallbackInfo(0, this, addinID);

    #region Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(cookie);
    AddCommandMgr();
    #endregion

    #region Setup the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    #region Setup Sample Property Manager
    AddPMP();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    RemoveCommandMgr();
    RemovePMP();
    DetachEventHandlers();

    iSwApp = null;
    //The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect();
    return true;
}

#endregion

#region UI Methods
public void AddCommandMgr()

```

```

{
    ICommandGroup cmdGroup;
    BitmapHandler iBmp = new BitmapHandler();
    Assembly thisAssembly;
    int cmdIndex0, cmdIndex1;
    string Title = "C# Add-in", ToolTip = "C# Add-in";

    int[] docTypes = new int[] {(int)swDocumentTypes_e.swDocASSEMBLY,
                                (int)swDocumentTypes_e.swDocDRAWING,
                                (int)swDocumentTypes_e.swDocPART};

    thisAssembly = System.Reflection.Assembly.GetAssembly(this.GetType());

    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1);
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.ToolbarLarge.bmp", thisAssembly);
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.ToolbarSmall.bmp", thisAssembly);
    cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.MainIconLarge.bmp", thisAssembly);
    cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.MainIconSmall.bmp", thisAssembly);

    cmdIndex0 = cmdGroup.AddCommandItem("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0);
    cmdIndex1 = cmdGroup.AddCommandItem("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP",
"EnablePMP", 2);

    cmdGroup.HasToolbar = true;
    cmdGroup.HasMenu = true;
    cmdGroup.Activate();

    bool bResult;

    foreach (int type in docTypes)
    {
        ICommandTab cmdTab;

        cmdTab = iCmdMgr.GetCommandTab(type, Title);

        if (cmdTab == null)
        {
            cmdTab = (ICommandTab)iCmdMgr.AddCommandTab(type, Title);

            CommandTabBox cmdBox = cmdTab.AddCommandTabBox();

            int[] cmdIDs = new int[3];
            int[] TextType = new int[3];

            cmdIDs[0] = cmdGroup.get_CommandID(cmdIndex0);
            System.Diagnostics.Debug.Print(cmdGroup.get_CommandID(cmdIndex0).ToString());
            TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

            cmdIDs[1] = cmdGroup.get_CommandID(cmdIndex1);
            System.Diagnostics.Debug.Print(cmdGroup.get_CommandID(cmdIndex1).ToString());
            TextType[1] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

            cmdIDs[2] = cmdGroup.ToolbarId;
            System.Diagnostics.Debug.Print(cmdIDs[2].ToString());
            TextType[2] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal |
(int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

            bResult = cmdBox.AddCommands(cmdIDs, TextType);

            CommandTabBox cmdBox1 = cmdTab.AddCommandTabBox();
            cmdIDs = new int[1];
            TextType = new int[1];

            cmdIDs[0] = cmdGroup.ToolbarId;
            TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow |
(int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

            bResult = cmdBox1.AddCommands(cmdIDs, TextType);

            cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId);

        }
    }

    // create a third-party icon in the context-sensitive menus of faces in parts
    frame = (IFrame)SwApp.Frame();
    resultCode = frame.AddMenuPopupIcon2((int)swDocumentTypes_e.swDocPART, (int)swSelectType_e.swSelFACES,
"contextsensitive", addinID, "CSCallbackFunction", "CSEnable", "", cmdGroup.SmallMainIcon);

    // create and register the third party menu
    registerID = SwApp.RegisterThirdPartyPopupMenu();

    // add a menu break at the top of the menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Menu Break", addinID,
"", "", "", "", "", (int)swMenuItemType_e.swMenuItemType_Break
);
    // add a couple of items to the menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Test1", addinID,
"TestCallback", "EnableTest", "", "Test1", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
);
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Test4", addinID,
"TestCallback", "EnableTest", "", "Test4", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
);
}

```

```

    );
    // add a separator bar to the menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "", addinID, "", "", "",
    "", "", (int)swMenuItemType_e.swMenuItemType_Separator);
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Test5", addinID,
    "TestCallback", "EnableTest", "", "Test5", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
    );

    // add an icon to the menu bar
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "", addinID,
    "TestCallback", "EnableTest", "", "NoOp", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
    );

    thisAssembly = null;
    iBmp.Dispose();
}

public void RemoveCommandMgr()
{
    iCmdMgr.RemoveCommandGroup(1);
}

public Boolean AddPMP()
{
    ppage = new UserPMPPage(this);
    return true;
}

public Boolean RemovePMP()
{
    ppage = null;
    return true;
}

#endregion

#region UI Callbacks
public void CreateCube()
{
    //make sure we have a part open
    string partTemplate = iSwApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
    IModelDoc2 modDoc = (IModelDoc2)iSwApp.NewDocument(partTemplate, (int)swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0);

    modDoc.InsertSketch2(true);
    modDoc.SketchRectangle(0, 0, 0, .1, .1, .1, false);
    //Extrude the sketch
    IFeatureManager featMan = modDoc.FeatureManager;
    featMan.FeatureExtrusion(true,
        false, false,
        (int)swEndConditions_e.swEndCondBlind, (int)swEndConditions_e.swEndCondBlind,
        0.1, 0.0,
        false, false,
        false, false,
        0.0, 0.0,
        false, false,
        false, false,
        true,
        false, false);
}

public void ShowPMP()
{
    if (ppage != null)
        ppage.Show();
}

public int EnablePMP()
{
    if (iSwApp.ActiveDoc != null)
        return 1;
    else
        return 0;
}

public void CSCallbackFunction()
{
    resultCode = SwApp.ShowThirdPartyPopupMenu(registerID, 500, 500);
}

public int CSEnable()
{
    if (iSwApp.ActiveDoc == null)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}

public void TestCallback()
{
    System.Diagnostics.Debug.Print("Test callback");
}

```

```

public int EnableTest()
{
    if (iSwApp.ActiveDoc == null)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}

#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    //Listen for events on all currently open docs
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        SwEventPtr.InterfaceBrightnessThemeChangeNotify += new
        DSldWorksEvents_InterfaceBrightnessThemeChangeNotifyEventHandler(InterfaceBrightnessThemeChangeNotify);

        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        SwEventPtr.InterfaceBrightnessThemeChangeNotify -= new
        DSldWorksEvents_InterfaceBrightnessThemeChangeNotifyEventHandler(InterfaceBrightnessThemeChangeNotify);

        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
            {
                docHandler = new PartEventHandler(modDoc, this);
            }
        }
    }
}

```

```

        break;
    }
    case (int)swDocumentTypes_e.swDocASSEMBLY:
    {
        docHandler = new AssemblyEventHandler(modDoc, this);
        break;
    }
    case (int)swDocumentTypes_e.swDocDRAWING:
    {
        docHandler = new DrawingEventHandler(modDoc, this);
        break;
    }
    default:
    {
        return false; //Unsupported document type
    }
}
docHandler.AttachEventHandlers();
openDocs.Add(modDoc, docHandler);
}
return true;
}

public bool DetachModelEventHandler(ModelDoc2 modDoc)
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

public bool DetachEventHandlers()
{
    DetachSwEvents();

    //Close events on all currently open docs
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    //Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        docHandler.DetachEventHandlers(); //This also removes the pair from the hash
        docHandler = null;
    }
    return true;
}
#endregion

#region Event Handlers
//Events
public int OnDocChange()
{
    return 0;
}

public int OnDocLoad(string docTitle, string docPath)
{
    return 0;
}

int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnFileNew(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnModelChange()
{
    return 0;
}

public int InterfaceBrightnessThemeChangeNotify(int theme, ref object colors)
{
    MessageBox.Show("Background changed to (0 = light, 1 = medium, 2 = dark, 3 = medium light): " + theme);
    int[] colorsArray = null;
    colorsArray = (int[])colors;
    MessageBox.Show("FeatureManager design tree color: " + colorsArray[0]);

    return 0;
}

```

```
} #endregion  
}
```

Add Shortcut Menus to Add-ins Example (VB.NET)

This example shows how to create a shortcut menu that launches from a third-party icon in the context-sensitive menus of part faces.

```

'-----
' Preconditions:
' 1. In Microsoft Visual Studio 2010, or later, create a SOLIDWORKS add-in
'    project using Visual Basic > SwVBAddin.
' 2. Name the project SwVBAddin1.
' 3. Copy and paste this code into SwAddin.vb of your VB.NET project.
' 4. Select Project > SwVBAddin1 Properties > Debug.
' 5. Select Start external program and type the path to sldworks.exe. Include
'    sldworks.exe in the path.
' 6. Press F5 to start debugging SOLIDWORKS Design with this add-in.
'
' Postconditions:
' 1. In SOLIDWORKS Design, click Tools > VB.NET Add-in > CreateCube.
' 2. Click a face on the cube.
' 3. Click the third-party icon in the face's context-sensitive menu.
' 4. Click a menu item on the shortcut menu and inspect the Immediate Window.
' 5. Click Tools > Options > System Options > Colors > select a different
'    Background > OK.
' 6. Displays the new background. Click OK to close the message box.
' 7. Displays the FeatureManager design tree decimal color value. Click OK
'    to close the message box.
'-----

```

```

' SwAddin.vb:

```

```

Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SOLIDWORKSTools
Imports SOLIDWORKSTools.File

```

```

<Guid("a3ddcdce-c8a8-47c2-8561-919a24285286")> _
<ComVisible(True)> _
<SwAddin( _
    Description:="SwVBAddin1 third-party pop-up menus", _
    Title:="SwVBAddin1", _
    LoadAtStartup:=True _
)> _
Public Class SwAddin
Implements SolidWorks.Interop.swpublished.SwAddin

```

```

#Region "Local Variables"
Dim WithEvents iSwApp As SldWorks
Dim iCmdMgr As ICommandManager
Dim addinID As Integer
Dim openDocs As Hashtable
Dim SwEventPtr As SldWorks
Dim ppage As UserPMPPage
Dim registerID As Integer
Dim resultCode As Boolean
Dim model As ModelDoc2
Dim frame As IFrame
Dim cmdGroup As ICommandGroup

' Public Properties
ReadOnly Property SwApp() As SldWorks
    Get
        Return iSwApp
    End Get
End Property

ReadOnly Property CmdMgr() As ICommandManager
    Get
        Return iCmdMgr
    End Get
End Property

ReadOnly Property OpenDocumentsTable() As Hashtable
    Get
        Return openDocs
    End Get
End Property
#End Region

```

```

#Region "SOLIDWORKS Registration"

```

```

<ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)

    ' Get Custom Attribute: SwAddinAttribute

```

```

Dim attributes() As Object
Dim SwAttr As SwAddinAttribute = Nothing

attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

If attributes.Length > 0 Then
    SwAttr = DirectCast(attributes(0), SwAddinAttribute)
End If

Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

Dim keyname As String = "SOFTWARE\SOLIDWORKS\Addins\" + t.GUID.ToString() + "}"
Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)
addinkey.SetValue(Nothing, 0)
addinkey.SetValue("Description", SwAttr.Description)
addinkey.SetValue("Title", SwAttr.Title)

keyname = "Software\SOLIDWORKS\AddInsStartup\" + t.GUID.ToString() + "}"
addinkey = hkcu.CreateSubKey(keyname)
addinkey.SetValue(Nothing, SwAttr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
End Sub

<ComUnregisterFunction(> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
    Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

    Dim keyname As String = "SOFTWARE\SOLIDWORKS\Addins\" + t.GUID.ToString() + "}"
    hklm.DeleteSubKey(keyname)

    keyname = "Software\SOLIDWORKS\AddInsStartup\" + t.GUID.ToString() + "}"
    hkcu.DeleteSubKey(keyname)
End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements
SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSwApp = ThisSW
    addinID = Cookie

    ' Setup callbacks
    iSwApp.SetAddinCallbackInfo(0, Me, addinID)

    ' Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(Cookie)
    AddCommandMgr()

    'Setup the Event Handlers
    SwEventPtr = iSwApp
    openDocs = New Hashtable
    AttachEventHandlers()

    'Setup Sample Property Manager
    AddPMP()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW

    RemoveCommandMgr()
    RemovePMP()
    DetachEventHandlers()

    iSwApp = Nothing
    'The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect()
    DisconnectFromSW = True
End Function
#End Region

#Region "UI Methods"

Public Sub AddCommandMgr()

    Dim iBmp As New BitmapHandler
    Dim thisAssembly As Assembly

    Dim cmdIndex0 As Integer, cmdIndex1 As Integer
    Dim Title As String = "VB.NET Add-in"
    Dim ToolTip As String = "VB.NET Add-in"

    Dim docTypes() As Integer = {swDocumentTypes_e.swDocASSEMBLY, _
        swDocumentTypes_e.swDocDRAWING, _
        swDocumentTypes_e.swDocPART}

    thisAssembly = System.Reflection.Assembly.GetAssembly(Me.GetType())

    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1)
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("SwVBAddin1.ToolbarLarge.bmp", thisAssembly)
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("SwVBAddin1.ToolbarSmall.bmp", thisAssembly)

```

```

cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("SwVbAddin1.MainIconLarge.bmp", thisAssembly)
cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("SwVbAddin1.MainIconSmall.bmp", thisAssembly)

cmdIndex0 = cmdGroup.AddCommandItem("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0)
cmdIndex1 = cmdGroup.AddCommandItem("Show PMP", -1, "Show PMP", "Show PMP", 2, "ShowPMP", "PMPEnable", 2)

cmdGroup.HasToolBar = True
cmdGroup.HasMenu = True
cmdGroup.Activate()

For Each docType As Integer In docTypes
    Dim cmdTab As ICommandTab = iCmdMgr.GetCommandTab(docType, Title)
    Dim bResult As Boolean

    If cmdTab Is Nothing Then
        cmdTab = iCmdMgr.AddCommandTab(docType, Title)

        Dim cmdBox As CommandTabBox = cmdTab.AddCommandTabBox

        Dim cmdIDs(3) As Integer
        Dim TextType(3) As Integer

        cmdIDs(0) = cmdGroup.CommandID(cmdIndex0)
        TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        cmdIDs(1) = cmdGroup.CommandID(cmdIndex1)
        TextType(1) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        cmdIDs(2) = cmdGroup.ToolbarId
        TextType(2) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        bResult = cmdBox.AddCommands(cmdIDs, TextType)

        Dim cmdBox1 As CommandTabBox = cmdTab.AddCommandTabBox()
        ReDim cmdIDs(1)
        ReDim TextType(1)

        cmdIDs(0) = cmdGroup.ToolbarId
        TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow

        bResult = cmdBox1.AddCommands(cmdIDs, TextType)

        cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId)
    End If
Next

' create a third-party icon in the context-sensitive menus of faces in parts
frame = SwApp.Frame()
resultCode = frame.AddMenuPopupIcon2(swDocumentTypes_e.swDocPART, swSelectType_e.swSelfFACES, "third-party context-sensitive", addinID, "CSCallbackFunction", "CSEnable", "", cmdGroup.SmallMainIcon)

' create and register the shortcut menu
registerID = SwApp.RegisterThirdPartyPopupMenu()
' add a menu break at the top of the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Menu Break", addinID, "", "", "", CInt(swMenuItemType_e.swMenuItemType_Break))
' add a couple of items to the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test1", addinID, "TestCallback", "EnableTest", "", "Test1", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test4", addinID, "TestCallback", "EnableTest", "", "Test4", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))
' add a separator bar to the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "separator", addinID, "", "", "", CInt(swMenuItemType_e.swMenuItemType_Separator))
' add another item to the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "Test5", addinID, "TestCallback", "EnableTest", "", "Test5", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))

' add an icon to a menu bar of the shortcut menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, CInt(swDocumentTypes_e.swDocPART), "", addinID, "TestCallback", "EnableTest", "", "NoOp", cmdGroup.SmallMainIcon, CInt(swMenuItemType_e.swMenuItemType_Default))

' remove an icon from the menu bar of the shortcut menu
' resultCode = SwApp.RemoveItemFromThirdPartyPopupMenu(registerID, CInt(swDocumentTypes_e.swDocPART), "", 1)
thisAssembly = Nothing
iBmp.Dispose()
End Sub

Public Sub RemoveCommandMgr()
    iCmdMgr.RemoveCommandGroup(1)
End Sub

Function AddPMP() As Boolean
    ppage = New UserPMPPage
    ppage.Init(iSwApp, Me)
End Function

Function RemovePMP() As Boolean
    ppage = Nothing
End Function
#End Region

```

```

#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()

    'Listen for events on all currently open docs
    AttachEventsToAllDocuments()
End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    'Close events on all currently open docs
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        'Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            docHandler.DetachEventHandlers() 'This also removes the pair from the hash
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
    AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
    AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
    AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
    AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    AddHandler iSwApp.InterfaceBrightnessThemeChangeNotify, AddressOf Me.SldWorks_InterfaceBrightnessChangeThemeNotify

End Sub

Sub DetachSWEvents()
    RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
    RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
    RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
    RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
    RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    RemoveHandler iSwApp.InterfaceBrightnessThemeChangeNotify, AddressOf Me.SldWorks_InterfaceBrightnessChangeThemeNotify

End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelDocEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

```

```

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_InterfaceBrightnessChangeThemeNotify(ByVal Theme As Integer, ByRef Colors as Object) As Integer
    MsgBox("Background changed to (0 = light, 1 = medium, 2 = dark, 3 = medium light): " & Theme)
    Dim colorsArray() As Integer
    colorsArray = Colors
    MsgBox("FeatureManager design tree color: " & colorsArray(0))
End Function
#End Region

#Region "UI Callbacks"
Sub CreateCube()

    'make sure we have a part open
    Dim partTemplate As String

    Dim featMan As FeatureManager

    partTemplate = iSwApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
    model = iSwApp.NewDocument(partTemplate, swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0)

    model.InsertSketch2(True)
    model.SketchRectangle(0, 0, 0, 0.1, 0.1, 0.1, False)

    'Extrude the sketch
    featMan = model.FeatureManager
    featMan.FeatureExtrusion(True, _
        False, False, _
        swEndConditions_e.swEndCondBlind, swEndConditions_e.swEndCondBlind, _
        0.1, 0.0, _
        False, False, _
        False, False, _
        0.0, 0.0, _
        False, False, _
        False, False, _
        True, _
        False, False)

End Sub
Sub CSCallbackFunction()
    resultCode = SwApp.ShowThirdPartyPopupMenu(registerID, 500, 500)
End Sub
Function CSEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        CSEnable = 0
    Else
        CSEnable = 1
    End If
End Function
Sub ShowPMP()
    ppage.Show()
End Sub

Function PMPEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        PMPEnable = 0
    Else
        PMPEnable = 1
    End If
End Function
Sub TestCallback()
    System.Diagnostics.Debug.Print("Test callback")
End Sub
Function EnableTest() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        EnableTest = 0
    Else
        EnableTest = 1
    End If
End Function
#End Region
End Class

```

Add Spotlight and Get Light Feature Example (C#)

This example shows how to add a spotlight to a model and get its light feature.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Adds a spotlight.
// 3. Gets the spotlight's feature and prints its ID to
// the Immediate window.
// 4. Examine the Immediate window, FeatureManager design tree, and
// graphics area.
//
// NOTE: Because this assembly document is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LightCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            ModelView swModelView = default(ModelView);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            Light swLight = default(Light);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int[] rect = null;

            //Open assembly
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\appearances\\usb_flash_drive1.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);

            //Add spotlight
            status = swModel.AddLightSource("Sw#5", 2, "Spot1");
            status = swModel.SetLightSourcePropertyValuesVB("Sw#5", 2, 0.5, 8454143, 1, -0.0999999999999999, 0.1700000000000001, 1.10999999999999, 0.179999
            99999999, -0.0999999999999999, 45, 0, 0, 0, 0.4, 0.4, 0, false);
            status = swModel.LockLightToModel(4, true);
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.GraphicsRedraw(rect);

            //Get spotlight's feature ID
            swModelDocExt = swModel.Extension;
            status = swModelDocExt.SelectByID2("Spot1", "LIGHTS", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swLight = (Light)swFeature.GetSpecificFeature2();
            Debug.Print("Light ID: " + swLight.GetID());
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Add Spotlight and Get Light Feature Example (VBA)

This example shows how to add a spotlight to a model and get its light feature.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Adds a spotlight.
' 3. Gets the spotlight's feature and prints its ID to
'    the Immediate window.
' 4. Examine the Immediate window, FeatureManager design tree, and
'    graphics area.
'
' NOTE: Because this assembly document is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swModelView As SldWorks.ModelView
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swLight As SldWorks.Light
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim rect As Variant

Sub main()

    Set swApp = Application.SldWorks

    'Open assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\appearances\usb_flash_drive1.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Add spotlight
    status = swModel.AddLightSource("Sw#5", 2, "Spot1")
    status = swModel.SetLightSourcePropertyValuesVB("Sw#5", 2, 0.5, 8454143, 1, -9.99999999999991E-02, 0.170000000000101, 1.10999999999999,
0.179999999999999, -9.00000000001008E-02, -1.02999999999999, 45, 0, 0, 0, 0.4, 0.4, 0, False)
    status = swModel.LockLightToModel(4, True)
    Set swModelView = swModel.ActiveView
    swModelView.GraphicsRedraw (rect)

    'Get spotlight's feature ID
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Spot1", "LIGHTS", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swLight = swFeature.GetSpecificFeature2
    Debug.Print "Light ID: " & swLight.GetID

End Sub
```

Add Spotlight and Get Light Feature Example (VB.NET)

This example shows how to add a spotlight to a model and get its light feature.

```

-----
Preconditions:
1. Verify that the specified assembly document exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified assembly document.
2. Adds a spotlight.
3. Gets the spotlight's feature and prints its ID to the Immediate window.
4. Examine the Immediate window, FeatureManager design tree, and graphics area.

NOTE: Because this assembly document is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swModelView As ModelView
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swLight As Light
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim rect() As Integer

        'Open assembly
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\appearances\usb_flash_drive1.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Add spotlight
        status = swModel.AddLightSource("SW#5", 2, "Spot1")
        status = swModel.SetLightSourcePropertyValuesVB("SW#5", 2, 0.5, 8454143, 1, -0.0999999999999999, 0.1700000000000001, 1.10999999999999, 0.1799999999
9999, -0.0900000000000000, -1.02999999999999, 45, 0, 0, 0.4, 0.4, 0, False)
        status = swModel.LockLightToModel(4, True)
        swModelView = swModel.ActiveView
        swModelView.GraphicsRedraw(rect)

        'Get spotlight's feature ID
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Spot1", "LIGHTS", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swLight = swFeature.GetSpecificFeature2
        Debug.Print("Light ID: " & swLight.GetID)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Add Spring to Motion Study Example (C#)

This example shows how to add a spring to a motion study.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\wrench.sldasm.
// 2. Verify that the MotionManager tab is visible. If it is not visible,
//    click View > MotionManager.
// 3. Right-click the project, select Add Reference, click Browse, and
//    select install_dir\api\redist\SolidWorks.Interop.swmotionstudy.dll.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Adds a spring feature between the grips of the wrench.
// 2. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SwMotionStudy;
using System;
using System.Diagnostics;
namespace ExampleCS.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            MotionStudyManager swMotionMgr;
            MotionStudy swMotionStudy1;
            SimulationSpringFeatureData swSpringFeat;
            bool boolstatus;
            Feature swFeat;
            SelectionMgr swSelMgr;

            swModel = swApp.ActiveDoc as ModelDoc2;
            swModelDocExt = swModel.Extension;
            swSelMgr = swModel.SelectionManager as SelectionMgr;

            // Get the MotionManager
            swMotionMgr = swModelDocExt.GetMotionStudyManager() as MotionStudyManager;
            if ((swMotionMgr == null))
            {
                return;
            }

            // Get "MotionStudy1_Distance=0.5in"
            swMotionStudy1 = swMotionMgr.GetMotionStudy("MotionStudy1_Distance=0.5in");
            if ((swMotionStudy1 == null))
            {
                swApp.SendMsgToUser("MotionStudy1_Distance=0.5in is not available.");
                return;
            }

            // Activate swMotionStudy1
            swMotionStudy1.Activate();

            // Define spring feature
            swSpringFeat = swMotionStudy1.CreateDefinition((int)swFeatureNameID_e.swFmAEMLinearMotionSpring) as
            SimulationSpringFeatureData;
            if (swSpringFeat == null)
```

```

    {
        Debug.Print("ERROR: Creation of Spring feature data object failed.");
        return;
    }

    // Select spring's endpoints
    Face2 swFace1;
    Face2 swFace2;
    swModel.ShowNamedView2("**Left", 3);
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.03344586330968, 0.0525345575174, 0, true, 0, null, 0);
    swFace1 = swSelMgr.GetSelectedObject6(1, -1) as Face2;
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.02244533711473, 0.0131288302002, 2.238961779386E-04, true, 0,
null,
0);

    swFace2 = swSelMgr.GetSelectedObject6(2, -1) as Face2;

    // Set spring's characteristics
    swSpringFeat.SetEndPoints(swFace1, swFace2);
    swSpringFeat.CoilDiameter = 0.0102;
    swSpringFeat.NumberOfCoils = 3;
    swSpringFeat.WireDiameter = 0.00152;
    swSpringFeat.FreeLength = 0.02;

    // Create Spring feature
    swFeat = swMotionStudy1.CreateFeature(swSpringFeat) as Feature;
    if (swFeat == null)
    {
        Debug.Print(" ERROR: Creation of the Spring feature failed.");
    }
    else
    {
        Debug.Print("Type of the feature added: " + swFeat.GetTypeName2());
    }

    Debug.Print("Type of spring as defined in swSpringType_e: " + swSpringFeat.Type.ToString());
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Add Spring to Motion Study Example (VBA)

This example shows how to add a spring to a motion study.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\wrench.sldasm.
' 2. Verify that the MotionManager tab is visible. If it is not visible,
'    click View > MotionManager.
' 3. Reference the SOLIDWORKS MotionStudy type library.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Adds a spring feature between the grips of the wrench.
' 2. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swMotionMgr As MotionStudyManager
    Dim swMotionStudy1 As MotionStudy
    Dim swSpringFeat As SimulationSpringFeatureData
    Dim boolstatus As Boolean
    Dim swFeat As SldWorks.Feature
    Dim swSelMgr As SelectionMgr

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    If (swMotionMgr Is Nothing) Then
        End
    End If

    ' Get and activate "MotionStudy1_Distance=0.5in"
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("MotionStudy1_Distance=0.5in")
    If (swMotionStudy1 Is Nothing) Then
        MsgBox "MotionStudy1_Distance=0.5in is not available."
        End
    End If

    ' Activate swMotionStudy1
    swMotionStudy1.Activate

    ' Define Spring feature
    Set swSpringFeat = swMotionStudy1.CreateDefinition(swFmAEMLinearMotionSpring)
    If swSpringFeat Is Nothing Then
        Debug.Print "ERROR: Creation of Spring feature data object failed."
        Exit Sub
    End If

    ' Select the faces for the spring's endpoints
    Dim swFace1 As face2, swFace2 As face2
    swModel.ShowNamedView2 "**Left", 3
    swModel.ViewZoomtofit2

```

```

boolstatus = swModelDocExt.SelectByID("", "FACE", 0.03344586330968, 0.0525345575174, 0, True, 0, Nothing, 0)
Set swFace1 = swSelMgr.GetSelectedObject6(1, -1)
boolstatus = swModelDocExt.SelectByID("", "FACE", 0.02244533711473, 0.0131288302002, 2.238961779386E-04, True, 0, Nothing, 0)
Set swFace2 = swSelMgr.GetSelectedObject6(2, -1)

' Set spring's characteristics
swSpringFeat.SetEndPoints swFace1, swFace2
swSpringFeat.CoilDiameter = 0.0102
swSpringFeat.NumberOfCoils = 3
swSpringFeat.WireDiameter = 0.00152
swSpringFeat.FreeLength = 0.02

' Create Spring feature
Set swFeat = swMotionStudy1.CreateFeature(swSpringFeat)

If swFeat Is Nothing Then
    Debug.Print " ERROR: Creation of the Spring feature failed."
Else
    Debug.Print "Type of the feature added: " & swFeat.GetTypeName2
End If

Debug.Print "Type of spring as defined in swSpringType_e: " & swSpringFeat.Type
End Sub

```

Add Spring to Motion Study Example (VB.NET)

This example shows how to add a spring to a motion study.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\wrench.sldasm.
' 2. Verify that the MotionManager tab is visible. If it is not visible,
'    click View > MotionManager.
' 3. Right-click the project, select Add Reference, click Browse, and
'    select install_dir\api\redist\SolidWorks.Interop.swmotionstudy.dll.
' 4. Open the Immediate window.
'
' Postconditions:
' 1 Adds a spring feature between the grips of the wrench.
' 2. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports SwMotionStudy
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swMotionMgr As MotionStudyManager
```

```
        Dim swMotionStudy1 As MotionStudy
```

```
        Dim swSpringFeat As SimulationSpringFeatureData
```

```
        Dim boolstatus As Boolean
```

```
        Dim swFeat As Feature
```

```
        Dim swSelMgr As SelectionMgr
```

```
        swModel = swApp.ActiveDoc
```

```
        swModelDocExt = swModel.Extension
```

```
        swSelMgr = swModel.SelectionManager
```

```
        ' Get the MotionManager
```

```
        swMotionMgr = swModelDocExt.GetMotionStudyManager()
```

```
        If (swMotionMgr Is Nothing) Then
```

```
            Exit Sub
```

```
        End If
```

```
        ' Get "MotionStudy1_Distance=0.5in"
```

```
        swMotionStudy1 = swMotionMgr.GetMotionStudy("MotionStudy1_Distance=0.5in")
```

```
        If (swMotionStudy1 Is Nothing) Then
```

```
            MsgBox("MotionStudy1_Distance=0.5in is not available.")
```

```
            Exit Sub
```

```
        End If
```

```
        ' Activate "MotionStudy1_Distance=0.5in"
```

```
        swMotionStudy1.Activate()
```

```
        ' Define spring feature
```

```
        swSpringFeat = swMotionStudy1.CreateDefinition(swFeatureNameID_e.swFmAEMLinearMotionSpring)
```

```
        If swSpringFeat Is Nothing Then
```

```
            Debug.Print("ERROR: Creation of Spring feature data object failed.")
```

```
            Exit Sub
```

```
        End If
```

```

' Select spring's endpoints
Dim swFace1 As Face2, swFace2 As Face2
swModel.ShowNamedView2("*Left", 3)
boolstatus = swModelDocExt.SelectByID("", "FACE", 0.03344586330968, 0.0525345575174, 0, True, 0, Nothing, 0)
swFace1 = swSelMgr.GetSelectedObject6(1, -1)
boolstatus = swModelDocExt.SelectByID("", "FACE", 0.02244533711473, 0.0131288302002, 0.0002238961779386, True, 0, Nothing,
0)

swFace2 = swSelMgr.GetSelectedObject6(2, -1)

' Set spring characteristics
swSpringFeat.SetEndPoint1(swFace1, swFace2)
swSpringFeat.CoilDiameter = 0.0102
swSpringFeat.NumberOfCoils = 3
swSpringFeat.WireDiameter = 0.00152
swSpringFeat.FreeLength = 0.02

' Create Spring feature
swFeat = swMotionStudy1.CreateFeature(swSpringFeat)
If swFeat Is Nothing Then
    Debug.Print(" ERROR: Creation of the Spring feature failed.")
Else
    Debug.Print("Type of the feature added: " & swFeat.GetTypeName2)
End If

Debug.Print("Type of spring as defined in swSpringType_e: " & swSpringFeat.Type)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Add Task Pane View Example (C#)

This example shows how to add a tab to the Task Pane.

```
//-----
// Preconditions:
// 1. Create bitmaps in 6 pixel sizes:
//    20 X 20
//    32 X 32
//    40 X 40
//    64 X 64
//    96 X 96
//    128 X 128
// 2. Replace the bitmap array elements in the macro to full pathnames of
//    these bitmap files.
//
// Postconditions:
// 1. Examine the Task Pane.
// 2. Displays a tab on the Task Pane with an image that is appropriate for
//    the current screen resolution or operating system scale.
//-----
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace AddTaskPaneView_CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            TaskpaneView ctrl;
            string[] bitmap = new string[6];
            string tooltip;

            bitmap[0] = "20x20_bitmap_pathname";
            bitmap[1] = "32x32_bitmap_pathname";
            bitmap[2] = "40x40_bitmap_pathname";
            bitmap[3] = "64x64_bitmap_pathname";
            bitmap[4] = "96x96_bitmap_pathname";
            bitmap[5] = "128x128_bitmap_pathname";
            tooltip = "";
            ctrl = swApp.CreateTaskpaneView3(bitmap, tooltip);

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>
            public SldWorks swApp;
        }
    }
}
```

Add Task Pane View Example (VBA)

This example shows how to add a tab to the Task Pane.

```
'-----  
' Preconditions:  
' 1. Create bitmaps in 6 pixel sizes:  
'    20 X 20  
'    32 X 32  
'    40 X 40  
'    64 X 64  
'    96 X 96  
'    128 X 128  
' 2. Replace the bitmap array elements in the macro to full pathnames of  
'    these bitmap files.  
'  
' Postconditions:  
' 1. Examine the Task Pane.  
' 2. Displays a tab on the Task Pane with an image that is appropriate for  
'    the current screen resolution or operating system scale.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim ctrl As SldWorks.TaskpaneView  
Dim bitmap(5) As String  
Dim toolTip As String
```

```
Sub main()  
    Set swApp = Application.SldWorks  
  
    bitmap(0) = "20x20_bitmap_pathname"  
    bitmap(1) = "32x32_bitmap_pathname"  
    bitmap(2) = "40x40_bitmap_pathname"  
    bitmap(3) = "64x64_bitmap_pathname"  
    bitmap(4) = "96x96_bitmap_pathname"  
    bitmap(5) = "128x128_bitmap_pathname"  
  
    toolTip = ""  
    Set ctrl = swApp.CreateTaskpaneView3(bitmap, toolTip)  
End Sub
```

Add Task Pane View Example (VB.NET)

This example shows how to add a tab to the Task Pane.

```

'-----
' Preconditions:
' 1. Create bitmaps in 6 pixel sizes:
'    20 X 20
'    32 X 32
'    40 X 40
'    64 X 64
'    96 X 96
'    128 X 128
' 2. Replace the bitmap array elements in the macro to full pathnames of
'    these bitmap files.
'
' Postconditions:
' 1. Examine the Task Pane.
' 2. Displays a tab on the Task Pane with an image that is appropriate for
'    the current screen resolution or operating system scale.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Class SolidWorksMacro
```

```

    Dim ctrl As TaskpaneView
    Dim bitmap(5) As String
    Dim toolTip As String

```

```
    Sub main()
```

```

        bitmap(0) = "20x20_bitmap_pathname"
        bitmap(1) = "32x32_bitmap_pathname"
        bitmap(2) = "40x40_bitmap_pathname"
        bitmap(3) = "64x64_bitmap_pathname"
        bitmap(4) = "96x96_bitmap_pathname"
        bitmap(5) = "128x128_bitmap_pathname"

```

```

        toolTip = ""
        ctrl = swApp.CreateTaskpaneView3(bitmap, toolTip)

```

```
    End Sub
```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

```
End Class
```

Add Third-party Application Keywords to SOLIDWORKS Search and Model Example (VBA)

This example shows how to add third-party application keywords to SOLIDWORKS Search and to a model document.

```
'-----
' Preconditions: Create a new drawing.
'
' Postconditions:
' 1. Adds the specified keyword and value to SOLIDWORKS Search
'   for the specified application and drawing.
' 2. Click Tools > Options > Search > Always index (may slow
'   SOLIDWORKS Design), if this check box is not selected.
' 3. Close the drawing and close SOLIDWORKS Design.
' 4. Restart SOLIDWORKS Design and open the drawing closed in step 3.
' 5. Run Get SOLIDWORKS Search Third-party Keywords Example \(VBA\).
' 6. Deselect the Always index (may slow SOLIDWORKS Design) check box
'   if the check box was previously not selected.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim longstatus As Long, longwarnings As Long
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Assign keywords and values
    boolstatus = swModelDocExt.AddOrUpdateSearchData("COSMOSWorks", "FirstName", "John")
    boolstatus = swModelDocExt.AddOrUpdateSearchData("COSMOSWorks", "MiddleName", "D")
    boolstatus = swModelDocExt.AddOrUpdateSearchData("COSMOSWorks", "LastName", "Doe")
    boolstatus = swModel.Save3(swSaveAsOptions_Silent, longstatus, longwarnings)

End Sub
```

Add Toolbars Example (C#)

This example shows how to use an add-in to add toolbars to a part document.

```
//-----
// Preconditions:
// 1. Verify that the SOLIDWORKS API SDK add-in templates for your version
//   of Microsoft Visual Studio are installed.
// 2. In Microsoft Visual Studio, create a C# project using the
//   SwCSharpAddin template.
// 3. Name the project AddToolbarCSharp.
// 4. Copy and paste SwAddin into SwAddin.cs of your project.
// 5. Click Project > AddToolbarCSharp Properties > Debug.
// 6. Select Start external program and type the pathname of your
//   SOLIDWORKS executable.
// 7. Replace the toolbarImages array elements with the pathnames of
//   your image files.
// 8. Press F5 to start debugging this add-in.
//
// Postconditions:
// 1. In SOLIDWORKS, click File > New > Part > OK.
// 2. Click the toolbar button located below the main menu.
// 3. Click OK to close the message box.
// 4. Click Tools > Customize.
//   a. Select a different icon size in Icon size.
//   b. Click OK three times.
//   c. Click the toolbar button located below the main menu.
//   d. Click OK to close the message box.
// 5. Repeat step 4, but select a different icon size for step 4a.
//-----
//SwAddin
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Collections.Generic;
using System.Diagnostics;
using System.Windows.Forms;

namespace AddToolbarCSharp
{
    /// <summary>
    /// Summary description for AddToolbarCSharp
    /// </summary>
    [Guid("d05b35ff-9a43-40ed-9c02-ebb886be116d"), ComVisible(true)]
    [SwAddin(
        Description = "AddToolbarCSharp description",
        Title = "AddToolbarCSharp",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        ISldWorks iSwApp = null;
        int addinID = 0;
        BitmapHandler iBmp;

        #region Event Handler Variables
        Hashtable openDocs = new Hashtable();
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr = null;
        #endregion

        // Public properties
        public ISldWorks SwApp
        {
            get { return iSwApp; }
        }

        public Hashtable OpenDocs
        {
            get { return openDocs; }
        }

        #endregion

        #region SolidWorks Registration
        [ComRegisterFunctionAttribute]
        public static void RegisterFunction(Type t)
        {
            #region Get Custom Attribute: SwAddinAttribute
            SwAddinAttribute SwAttr = null;
            Type type = typeof(SwAddin);

            foreach (System.Attribute attr in type.GetCustomAttributes(false))
            {
                if (attr is SwAddinAttribute)
                {
                    SwAttr = attr as SwAddinAttribute;
                    break;
                }
            }

            #endregion

            try

```

```

    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\{" + t.GUID.ToString() + "}";
        Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
        addinkey.SetValue(null, 0);

        addinkey.SetValue("Description", Swattr.Description);
        addinkey.SetValue("Title", Swattr.Title);

        keyname = "Software\\SolidWorks\\AddInsStartup\\{" + t.GUID.ToString() + "}";
        addinkey = hkcu.CreateSubKey(keyname);
        addinkey.SetValue(null, Convert.ToInt32(Swattr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem registering this dll: Swattr is null. \n\" + nl.Message + "\"");
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: Swattr is null. \n\" + nl.Message + "\"");
    }

    catch (System.Exception e)
    {
        Console.WriteLine(e.Message);

        System.Windows.Forms.MessageBox.Show("There was a problem registering the function: \n\" + e.Message + "\"");
    }
}

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction(Type t)
{
    try
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\{" + t.GUID.ToString() + "}";
        hklm.DeleteSubKey(keyname);

        keyname = "Software\\SolidWorks\\AddInsStartup\\{" + t.GUID.ToString() + "}";
        hkcu.DeleteSubKey(keyname);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + nl.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\" + nl.Message + "\"");
    }
    catch (System.Exception e)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + e.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\" + e.Message + "\"");
    }
}

#endregion

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    // Set up callbacks
    iSwApp.SetAddinCallbackInfo2(0, this, addinID);

    #region Set up the toolbar
    AddToolBar();
    #endregion

    #region Set up the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    DetachEventHandlers();

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSwApp);
    iSwApp = null;
    // The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect();
    GC.WaitForPendingFinalizers();

    GC.Collect();
    GC.WaitForPendingFinalizers();

    return true;
}
#endregion

#region UI Methods

// Add toolbar using ISldWorks::AddToolBar5 method
public void AddToolBar()
{
    Assembly thisAssembly1;
    string[] toolbarImages = new string[3];
    bool bret = false;

```

```

        int iToolBarId = 0;

        thisAssembly1 = System.Reflection.Assembly.GetAssembly(this.GetType());

        toolbarImages[0] = "Pathname_to_toolbar_nxn_image";
        toolbarImages[1] = "Pathname_to_toolbar_nnnxn_image";
        toolbarImages[2] = "Pathname_to_toolbar_nnnnnxn_image";

        iToolBarId = iSwApp.AddToolBar5(addinID, "Test Toolbar", toolbarImages, 0, (int)swDocTemplateTypes_e.swDocTemplateTypePART);

        bret = iSwApp.AddToolBarCommand2(addinID, iToolBarId, 0, "ButtonCallback", "ButtonEnableMethod", "Test toolbar ToolTip", "Hint string for test
toolbar");
    }

    public void ButtonCallback()
    {
        MessageBox.Show("Button callback function called.");
    }

    public int ButtonEnableMethod()
    {
        return 1;
    }

    public bool CompareIDs(int[] storedIDs, int[] addinIDs)
    {
        List<int> storedList = new List<int>(storedIDs);
        List<int> addinList = new List<int>(addinIDs);

        addinList.Sort();
        storedList.Sort();

        if (addinList.Count != storedList.Count)
        {
            return false;
        }
        else
        {
            for (int i = 0; i < addinList.Count; i++)
            {
                if (addinList[i] != storedList[i])
                {
                    return false;
                }
            }
        }
        return true;
    }

#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    // Listen for events on all currently open documents
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(ActiveDocChangeNotify);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(DocumentLoadNotify2);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(FileNewNotify2);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(ActiveModelDocChangeNotify);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        SwEventPtr.FileOpenPreNotify += new DSldWorksEvents_FileOpenPreNotifyEventHandler(FileOpenPreNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(ActiveDocChangeNotify);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(DocumentLoadNotify2);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(FileNewNotify2);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(ActiveModelDocChangeNotify);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        SwEventPtr.FileOpenPreNotify -= new DSldWorksEvents_FileOpenPreNotifyEventHandler(FileOpenPreNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))

```

```

        {
            AttachModelDocEventHandler(modDoc);
        }
        else if (openDocs.Contains(modDoc))
        {
            bool connected = false;
            DocumentEventHandler docHandler = (DocumentEventHandler)openDocs[modDoc];
            if (docHandler != null)
            {
                connected = docHandler.ConnectModelViews();
            }
        }

        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
            {
                docHandler = new PartEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocASSEMBLY:
            {
                docHandler = new AssemblyEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocDRAWING:
            {
                docHandler = new DrawingEventHandler(modDoc, this);
                break;
            }
            default:
            {
                // Unsupported document type
                return false;
            }
        }
        docHandler.AttachEventHandlers();
        openDocs.Add(modDoc, docHandler);
    }
    return true;
}

public bool DetachModelEventHandler(ModelDoc2 modDoc)
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

public bool DetachEventHandlers()
{
    DetachSwEvents();

    // Close events on all currently open documents
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    // Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        // Remove the pair from the hash
        docHandler.DetachEventHandlers();
        docHandler = null;
    }
    return true;
}
#endregion

#region Event Handlers
//Events
public int ActiveDocChangeNotify()
{
    return 0;
}

public int DocumentLoadNotify2(string docTitle, string docPath)
{
    return 0;
}

int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

int FileOpenPreNotify(String FileName)
{
    return 0;
}

```

```
public int FileNewNotify2(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int ActiveModelDocChangeNotify()
{
    return 0;
}

#endregion
}
```

Add Toolbars Example (VB.NET)

This example shows how to use an add-in to add toolbars to a part document.

```

'-----
' Preconditions:
' 1. Verify that the SOLIDWORKS API SDK add-in templates for your version
'   of Microsoft Visual Studio are installed.
' 2. In Microsoft Visual Studio, create a Visual Basic project using the
'   SwVBAddin template.
' 3. Name the project AddToolbarVBNET.
' 4. Copy and paste SwAddin into SwAddin.vb of your project.
' 5. Click Project > AddToolbarVBNET Properties > Debug.
' 6. Select Start external program and type the pathname of your
'   SOLIDWORKS executable.
' 7. Replace the toolbarImages array elements with the pathnames of
'   your image files.
' 8. Press F5 to start debugging this add-in.
'
' Postconditions:
' 1. In SOLIDWORKS, click File > New > Part > OK.
' 2. Click the toolbar button located below the main menu.
' 3. Click OK to close the message box.
' 4. Click Tools > Customize.
'   a. Select a different icon size in Icon size.
'   b. Click OK three times.
'   c. Click the toolbar button located below the main menu.
'   d. Click OK to close the message box.
' 5. Repeat step 4, but select a different icon size for step 4a.
'-----
'SwAddin

Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SolidWorksTools
Imports SolidWorksTools.File
Imports System.Windows.Forms

Imports System.Collections.Generic
Imports System.Diagnostics

<Guid("606dd567-4659-4c66-ae88-2fc20ef35891")> _
<ComVisible(True)> _
<SwAddin( _
    Description:="AddToolbarVBNET description", _
    Title:="AddToolbarVBNET", _
    LoadAtStartup:=True _
)> _
Public Class SwAddin
Implements SolidWorks.Interop.swpublished.SwAddin

#Region "Local Variables"
Dim WithEvents iSwApp As SldWorks
Dim addinID As Integer
Dim openDocs As Hashtable
Dim SwEventPtr As SldWorks

' Public properties
ReadOnly Property SwApp() As SldWorks
Get
    Return iSwApp
End Get
End Property

ReadOnly Property OpenDocumentsTable() As Hashtable
Get
    Return openDocs
End Get
End Property
#End Region

#Region "SolidWorks Registration"

<ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)
' Get Custom Attribute: SwAddinAttribute
Dim attributes() As Object
Dim SwAttr As SwAddinAttribute = Nothing

attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

If attributes.Length > 0 Then
    SwAttr = DirectCast(attributes(0), SwAddinAttribute)
End If
Try
    Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
    Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

    Dim keyname As String = "SOFTWARE\SolidWorks\Addins\" + t.GUID.ToString() + "}"
    Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)
    addinkey.SetValue(Nothing, 0)
    addinkey.SetValue("Description", SwAttr.Description)
    addinkey.SetValue("Title", SwAttr.Title)

    keyname = "Software\SolidWorks\AddInsStartup\" + t.GUID.ToString() + "}"
    addinkey = hkcu.CreateSubKey(keyname)
    addinkey.SetValue(Nothing, SwAttr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
Catch nl As System.NullReferenceException
    Console.WriteLine("There was a problem registering this dll: SwAttr is null.\n " & nl.Message)
    System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: SwAttr is null.\n" & nl.Message)
Catch e As System.Exception
    Console.WriteLine("There was a problem registering this dll: " & e.Message)
    System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: " & e.Message)

```

```

    End Try
End Sub

<ComUnregisterFunction()> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Try
        Dim hkLM As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkCU As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SolidWorks\Addins\{" + t.GUID.ToString() + "}"
        hkLM.DeleteSubKey(keyname)

        keyname = "Software\SolidWorks\AddInsStartup\{" + t.GUID.ToString() + "}"
        hkCU.DeleteSubKey(keyname)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem unregistering this dll: Swatrr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: Swatrr is null.\n" & nl.Message)
    Catch e As System.Exception
        Console.WriteLine("There was a problem unregistering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: " & e.Message)
    End Try
End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSWApp = ThisSW
    addinID = Cookie

    ' Set up callbacks
    iSWApp.SetAddinCallbackInfo2(0, Me, addinID)

    ' Set up the toolbar
    AddToolBar()

    ' Set up the Event Handlers
    SwEventPtr = iSWApp
    openDocs = New Hashtable
    AttachEventHandlers()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW
    DetachEventHandlers()

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSWApp)
    iSWApp = Nothing
    'The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect()
    GC.WaitForPendingFinalizers()

    GC.Collect()
    GC.WaitForPendingFinalizers()

    DisconnectFromSW = True
End Function
#End Region

#Region "UI Methods"
' Add toolbar using ISldWorks::AddToolBar5 method
Public Sub AddToolBar()
    Dim thisAssembly1 As Assembly
    Dim toolbarImages As String() = New String(2) {}
    Dim bret As Boolean = False
    Dim iToolBarId As Integer = 0

    thisAssembly1 = System.Reflection.Assembly.GetAssembly(Me.GetType())

    toolbarImages(0) = "Pathname_to_toolbar_nxn_image"
    toolbarImages(1) = "Pathname_to_toolbar_nnxnn_image"
    toolbarImages(2) = "Pathname_to_toolbar_nnnxnnn_image"

    iToolBarId = iSWApp.AddToolBar5(addinID, "Test Toolbar", toolbarImages, 0, CInt(swDocTemplateTypes.e.swDocTemplateTypePART))

    bret = iSWApp.AddToolBarCommand2(addinID, iToolBarId, 0, "ButtonCallback", "ButtonEnableMethod", "Test toolbar ToolTip", _
        "Hint string for test toolbar")
End Sub

Public Sub ButtonCallback()
    MessageBox.Show("Button callback function called.")
End Sub

Function ButtonEnableMethod() As Boolean
    ' TODO: Add your implementation here
End Function

Function CompareIDs(ByVal storedIDs() As Integer, ByVal addinIDs() As Integer) As Boolean
    Dim storeList As New List(Of Integer)(storedIDs)
    Dim addinList As New List(Of Integer)(addinIDs)

    addinList.Sort()
    storeList.Sort()

    If Not addinList.Count = storeList.Count Then
        Return False
    Else
        For i As Integer = 0 To addinList.Count - 1
            If Not addinList(i) = storeList(i) Then
                Return False
            End If
        Next
    End If

    Return True
End Function
#End Region

#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()
End Sub

```

```

        'Listen for events on all currently open documents
AttachEventsToAllDocuments()
End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    'Close events on all currently open documents
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        'Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            'Remove the pair from the hash
            docHandler.DetachEventHandlers()
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    Try
        AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub DetachSWEvents()
    Try
        RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        Else
            Dim docHandler As DocumentEventHandler = openDocs(modDoc)
            If Not docHandler Is Nothing Then
                docHandler.ConnectModelViews()
            End If
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelDocEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function
#End Region

```

End Class

Add User-specified Type Library Reference Example (VBA)

This example shows how to add a user-specified type library reference.

```
'-----  
' Preconditions: User-specified type library exists.  
'  
' Postconditions: User-specified type library is added. It is  
'                visible on VBA Available References list  
'                only after recording a macro.  
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim v1 As Variant

Dim s(0) As String

Sub main()

```
    Set swApp = Application.SldWorks  
    s(0) = "c:\Program Files\MyAddin\myaddin.tlb"  
    v1 = s  
    swApp.UserTypeLibReferences = v1  
    Debug.Print v1(0)
```

End Sub

Add Watermark to Part Example (C#)

This example shows how to add and modify a watermark.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Insert a note for a watermark.
//    a. Expand the Annotations folder in the FeatureManager
//       design tree.
//    b. Right click Notes Area and click Activate.
//    c. Click Insert > Annotations > Note.
//    d. Place the note on the model, type the note text,
//       and click OK in the Notes PropertyManager page.
//    e. Right-click the note and click Watermark.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Places the watermark behind the geometry and
//    sets its transparency level to 50%.
// 2. Examine the graphics area and the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Note swNote = default(Note);
            SelectionMgr swSelectionMgr = default(SelectionMgr);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swNote = (Note)swSelectionMgr.GetSelectedObject6(1, 0);
            Debug.Print("Is note a watermark? " + swNote.WatermarkNote);
            swNote.WatermarkBehindGeometry = true;
            Debug.Print("Is note behind geometry? " + swNote.WatermarkBehindGeometry);
            swNote.WatermarkTransparencyLevel = 0.50;
            Debug.Print("Note transparency level = " + swNote.WatermarkTransparencyLevel * 100 + "%");

            swModel.ClearSelection2(true);
            swModel.EditRebuild3();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Add Watermark to Part Example (VBA)

This example shows how to add and modify a watermark.

```
'-----  
' Preconditions:  
' 1. Open a part document.  
' 2. Insert a note for a watermark.  
'   a. Expand the Annotations folder in the FeatureManager  
'      design tree.  
'   b. Right-click Notes Area and click Activate.  
'   c. Click Insert > Annotations > Note.  
'   d. Place the note on the model, type the note text, and  
'      click OK in the Note PropertyManager page.  
'   e. Right-click the note and click Watermark.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Places the watermark behind the geometry and  
'    sets its transparency level to 50%.  
' 2. Examine the graphics area and the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swNote As SldWorks.Note  
Dim swSelectionMgr As SldWorks.SelectionMgr  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swNote = swSelectionMgr.GetSelectedObject6(1, 0)  
    Debug.Print "Is note a watermark? " & swNote.WatermarkNote  
    swNote.WatermarkBehindGeometry = True  
    Debug.Print "Is note behind geometry? " & swNote.WatermarkBehindGeometry  
    swNote.WatermarkTransparencyLevel = 0.5  
    Debug.Print "Note transparency level = " & swNote.WatermarkTransparencyLevel * 100 & "%"  
  
    swModel.ClearSelection2 True  
    swModel.EditRebuild3  
  
End Sub
```

Add Watermark to Part Example (VB.NET)

This example shows how to add and modify a watermark.

```
-----
Preconditions:
1. Open a part document.
2. Insert a note for a watermark.
   a. Expand the Annotations folder in the FeatureManager design tree.
   b. Right-click Notes Area and click Activate.
   c. Click Insert > Annotations > Note.
   d. Place the note on the model, type the note text, and click OK in the Note PropertyManager page.
   e. Right-click the note and click Watermark.
3. Open the Immediate window.
Postconditions:
1. Places the watermark behind the geometry and sets its transparency level to 50%.
2. Examine the graphics area and the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swNote As Note
        Dim swSelectionMgr As SelectionMgr

        swModel = swApp.ActiveDoc
        swSelectionMgr = swModel.SelectionManager
        swNote = swSelectionMgr.GetSelectedObject6(1, 0)
        Debug.Print("Is note a watermark? " & swNote.WatermarkNote)
        swNote.WatermarkBehindGeometry = True
        Debug.Print("Is note behind geometry? " & swNote.WatermarkBehindGeometry)
        swNote.WatermarkTransparencyLevel = 0.5
        Debug.Print("Note transparency level = " & swNote.WatermarkTransparencyLevel * 100 & "%")

        swModel.ClearSelection2(True)
        swModel.EditRebuild3()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Align Assembly Component to Assembly Origin and Planes Example (VBA)

This example shows how to align an assembly component to the assembly origin and planes.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Select an assembly component.  
'  
' Postconditions:  
' 1. Aligns the selected assembly component  
'    to the assembly origin and planes.  
' 2. Examine the graphics area.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swMathUtil As SldWorks.MathUtility  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swComp As SldWorks.Component2  
    Dim swXform As SldWorks.MathTransform  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swMathUtil = swApp.GetMathUtility  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)  
    ' Create unit transform  
    Set swXform = swMathUtil.CreateTransform(Nothing)  
    swComp.Transform2 = swXform  
  
    ' Rebuild to see updated transform  
    bRet = swModel.EditRebuild3  
  
End Sub
```

Align Auxiliary Drawing View Example (C#)

This example shows how to align an auxiliary drawing view.

```
//-----
// Preconditions:
// 1. Open a drawing with an auxiliary view.
// 2. Modify this macro to select the auxiliary view.
//
// Postconditions:
// 1. Aligns the auxiliary view horizontally in the clockwise direction.
// 2. Press F5.
// 3. Aligns the auxiliary view horizontally in a counterclockwise direction.
// 4. Press F5.
// 5. Aligns the auxiliary view using the default alignment.
// 6. Press F5.
// 7. Aligns the auxiliary view along the projection angle.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace AlignAuxDrawView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = Part.Extension.SelectByID("Drawing View2", "DRAWINGVIEW", 0.110297569919827, 0.215100510639068, 0, false,
0, null, 0);
            View DrawView = default(View);
            DrawView = (View)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(1, -1);
            boolstatus = DrawView.AlignDrawingView((int)swAlignDrawingViewTypes_e.swHorizontalToSheetClockwise);
            System.Diagnostics.Debugger.Break();
            boolstatus = DrawView.AlignDrawingView((int)swAlignDrawingViewTypes_e.swHorizontalToSheetCounterclockwise);
            System.Diagnostics.Debugger.Break();
            boolstatus = DrawView.AlignDrawingView((int)swAlignDrawingViewTypes_e.swDefaultAlignment);
            System.Diagnostics.Debugger.Break();
            boolstatus = DrawView.AlignDrawingView((int)swAlignDrawingViewTypes_e.swProjectedAngle);
        }

        public SldWorks swApp;
    }
}
```

Align Auxiliary Drawing View Example (VBA)

This example shows how to align an auxiliary drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing with an auxiliary view.  
' 2. Modify this macro to select the auxiliary view.  
'  
' Postconditions:  
' 1. Aligns the auxiliary view horizontally in the clockwise direction.  
' 2. Press F5.  
' 3. Aligns the auxiliary view horizontally in a counterclockwise direction.  
' 4. Press F5.  
' 5. Aligns the auxiliary view using the default alignment.  
' 6. Press F5.  
' 7. Aligns the auxiliary view along the projection angle.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim boolstatus As Boolean
```

```
Sub main()  
    Set swApp = Application.SldWorks  
  
    Set Part = swApp.ActiveDoc  
    boolstatus = Part.Extension.SelectByID2("Drawing View2", "DRAWINGVIEW", 0.110297569919827, 0.215100510639068, 0, False, 0,  
Nothing, 0)  
    Dim DrawView As SldWorks.View  
    Set DrawView = Part.SelectionManager.GetSelectedObject6(1, -1)  
    boolstatus = DrawView.AlignDrawingView(swHorizontalToSheetClockwise)  
    Stop  
    boolstatus = DrawView.AlignDrawingView(swHorizontalToSheetCounterclockwise)  
    Stop  
    boolstatus = DrawView.AlignDrawingView(swDefaultAlignment)  
    Stop  
    boolstatus = DrawView.AlignDrawingView(swProjectedAngle)  
End Sub
```

Align Auxiliary Drawing View Example (VB.NET)

This example shows how to align an auxiliary drawing view.

```

'-----
' Preconditions:
' 1. Open a drawing with an auxiliary view.
' 2. Modify this macro to select the auxiliary view.
'
' Postconditions:
' 1. Aligns the auxiliary view horizontally in the clockwise direction.
' 2. Press F5.
' 3. Aligns the auxiliary view horizontally in a counterclockwise direction.
' 4. Press F5.
' 5. Aligns the auxiliary view using the default alignment.
' 6. Press F5.
' 7. Aligns the auxiliary view along the projection angle.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```

Partial Class SolidWorksMacro

```

```

    Dim Part As ModelDoc2
    Dim boolstatus As Boolean

```

```

    Sub main()

```

```

        Part = swApp.ActiveDoc
        boolstatus = Part.Extension.SelectByID2("Drawing View2", "DRAWINGVIEW", 0.110297569919827, 0.215100510639068, 0, False, 0,
Nothing, 0)
        Dim DrawView As View
        DrawView = Part.SelectionManager.GetSelectedObject6(1, -1)
        boolstatus = DrawView.AlignDrawingView(swAlignDrawingViewTypes_e.swHorizontalToSheetClockwise)
        Stop
        boolstatus = DrawView.AlignDrawingView(swAlignDrawingViewTypes_e.swHorizontalToSheetCounterclockwise)
        Stop
        boolstatus = DrawView.AlignDrawingView(swAlignDrawingViewTypes_e.swDefaultAlignment)
        Stop
        boolstatus = DrawView.AlignDrawingView(swAlignDrawingViewTypes_e.swProjectedAngle)

```

```

    End Sub

```

```

    Public swApp As SldWorks

```

```

End Class

```

Align Drawing Views Example (C#)

This example shows how to align drawing views.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing.
// 2. Horizontally aligns Drawing View6 with
//    the center of Drawing View3.
// 3. Examine the drawing and Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace AlignWithViewCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDrawingDoc;
        View swView;
        View swViewAlignWith;
        ModelDocExtension swModelDocExtension;
        SelectionMgr swSelectionMgr;
        bool status;
        int errors;
        int warnings;
        string fileName;

        public void Main()
        {
            // Open drawing
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawingDoc = (DrawingDoc)swModel;

            // Activate Sheet2
            status = swDrawingDoc.ActivateSheet("Sheet2");
            swModel.ClearSelection2(true);

            // Activate Drawing View6 and make it the active
            // drawing view
            status = swDrawingDoc.ActivateView("Drawing View6");
            swView = (View)swDrawingDoc.ActiveDrawingView;
            swModelDocExtension = (ModelDocExtension)swModel.Extension;

            // Select Drawing View3 and select it to be
            // the base view
            status = swModelDocExtension.SelectByID2("Drawing View3", "DRAWINGVIEW", 0.166130896593432, 0.265618781363279, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swViewAlignWith = (View)swSelectionMgr.GetSelectedObject6(1, -1);

            // Drawing View6 aligns horizontally
            // with the center of Drawing View3
            status = swView.AlignWithView((int)swAlignViewTypes_e.swAlignViewHorizontalCenter, swViewAlignWith);

            Debug.Print("Did the specified drawing views align? " + status);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Align Drawing Views Example (VBA)

This example shows how to align drawing views.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Horizontally aligns Drawing View6 with
'    the center of Drawing View3.
' 3. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swViewAlignWith As SldWorks.View
Dim swModelDocExtension As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim status As Boolean
Dim errors As Long, warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks

    ' Open drawing
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawingDoc = swModel

    ' Activate Sheet2
    status = swDrawingDoc.ActivateSheet("Sheet2")
    swModel.ClearSelection2 True

    ' Activate Drawing View6 and make it the active
    ' drawing view
    status = swDrawingDoc.ActivateView("Drawing View6")
    Set swView = swDrawingDoc.ActiveDrawingView
    Set swModelDocExtension = swModel.Extension

    ' Select Drawing View3 and select it to be
    ' the base view
    status = swModelDocExtension.SelectByID2("Drawing View3", "DRAWINGVIEW", 0.166130896593432, 0.265618781363279, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swViewAlignWith = swSelectionMgr.GetSelectedObject6(1, -1)

    ' Drawing View6 aligns horizontally
    ' with the center of Drawing View3
    status = swView.AlignWithView(swAlignViewHorizontalCenter, swViewAlignWith)

    Debug.Print ("Did the specified drawing views align? " & status)

End Sub
```

Align Drawing Views Example (VB.NET)

This example shows how to align drawing views.

```

-----
Preconditions:
1. Verify that the specified drawing document to open exists.
2. Open the Immediate window.
-----
Postconditions:
1. Opens the specified drawing.
2. Horizontally aligns Drawing View6 with
   the center of Drawing View3.
3. Examine the drawing and Immediate window.
-----
NOTE: Because the drawing is used elsewhere, do not
save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swDrawingDoc As DrawingDoc
    Dim swView As View
    Dim swViewAlignWith As View
    Dim swModelDocExtension As ModelDocExtension
    Dim swSelectionMgr As SelectionMgr
    Dim status As Boolean
    Dim errors As Integer, warnings As Integer
    Dim fileName As String

    Sub main()

        ' Open drawing
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawingDoc = swModel

        ' Activate Sheet2
        status = swDrawingDoc.ActivateSheet("Sheet2")
        swModel.ClearSelection2(True)

        ' Activate Drawing View6 and make it the active
        ' drawing view
        status = swDrawingDoc.ActivateView("Drawing View6")
        swView = swDrawingDoc.ActiveDrawingView
        swModelDocExtension = swModel.Extension

        ' Select Drawing View3 and select it to be
        ' the base view
        status = swModelDocExtension.SelectByID2("Drawing View3", "DRAWINGVIEW", 0.166130896593432, 0.265618781363279, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swViewAlignWith = swSelectionMgr.GetSelectedObject6(1, -1)

        ' Drawing View6 aligns horizontally
        ' with the center of Drawing View3
        status = swView.AlignWithView(swAlignViewTypes_e.swAlignViewHorizontalCenter, swViewAlignWith)

        Debug.Print("Did the specified drawing views align? " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Align Text with Line Example (VBA)

This example shows how to align text with a selected line.

```
'-----  
' Preconditions: A model document is open.  
'  
' Postconditions: A sketch containing a line is  
'               created on the Front plane and  
'               the word TEST is aligned with  
'               and created with that line.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchMgr As SldWorks.SketchManager  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
    Set swSketchMgr = swModel.SketchManager  
  
    ' Select the Front plane  
    boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
    ' Open a sketch  
    swSketchMgr.InsertSketch True  
    swModel.ClearSelection2 True  
  
    ' Create a line  
    swModel.CreateLine2 -0.04180198735321, 0.03305974480578, 0, 0.03898839205059, -0.02461895889792, 0  
    swModel.ClearSelection2 True  
  
    ' Rebuild and close the sketch  
    swSketchMgr.InsertSketch True  
    swModel.ClearSelection2 True  
  
    ' Select the Front plane  
    boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
    ' Open a sketch  
    swSketchMgr.InsertSketch True  
    swModel.ClearSelection2 True  
  
    ' Select the previously created line  
    boolstatus = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.01503940368923, -0.008275270054597, 0, False, 1, Nothing, 0)  
    ' Align and insert the word TEST on the selected line  
    swModel.InsertSketchText 0, 0, 0, "TEST", 0, 0, 0, 100, 100  
    swModel.ClearSelection2 True  
  
End Sub
```

Analyze Text and Geometry in GTol Symbol Example (C#)

This example shows how to analyze the text and geometry in a GTol symbol.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Gets whether <"GTOL-SPROF"> is bound.
// 3. Gets the number of lines, arcs, circles, text strings,
//    and triangles in <"GTOL-SPROF">.
// 4. Gets the number of point coordinates in the
//    lines in <"GTOL-SPROF">.
// 5. Gets the x,y,z coordinates for the first and
//    second points of each line in <"GTOL-SPROF">.
// 6. Gets the arc information for <"GTOL-SPROF">.
// 7. Examine the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetGTolSymbolInfo_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SolidWorks.Interop.sldworks.Environment swEnvironment;
        string fileName;
        int errors;
        int warnings;
        short[] nbrSymbols;
        double[] lineSymbols;
        double[] arcInfo;
        int nbrLineCoordinates;
        int i;

        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\tolanalyst\\minimum_clearance\\top_plate.sldprt";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "");
            swEnvironment = (SolidWorks.Interop.sldworks.Environment)swApp.GetEnvironment();

            Debug.Print("GTOL library - Surface Profile symbol name:");

            //Get whether <"GTOL-SPROF"> is bound
            bool bound;
            swEnvironment.GetIsBound("<GTOL-SPROF>", out bound);
            Debug.Print(" Bound? " + bound);
            Debug.Print("");

            //Get the number of lines, arcs, circles, text strings,
            //and triangles in <"GTOL-SPROF">
            nbrSymbols = (short[])swEnvironment.GetSymEdgeCounts("<GTOL-SPROF>");
            Debug.Print(" Line count: " + nbrSymbols[0]);
            Debug.Print(" Arc count: " + nbrSymbols[1]);
            Debug.Print(" Circle count: " + nbrSymbols[2]);
            Debug.Print(" Text count: " + nbrSymbols[3]);
            Debug.Print(" Triangle count: " + nbrSymbols[4]);
            Debug.Print("");
            Debug.Print("Number of lines: " + nbrSymbols[0]);

            //Get the x,y,z coordinates for the first
            //and second points of each line
            lineSymbols = (double[])swEnvironment.GetSymLines("<GTOL-SPROF>");
            nbrLineCoordinates = lineSymbols.GetUpperBound(0);
            Debug.Print(" Number of point coordinates: " + (nbrLineCoordinates + 1));
            Debug.Print(" Point coordinates:");
            Debug.Print(" (Line 1's x,y,z coordinates for first and second points,");
            Debug.Print(" Line 2's x,y,z coordinates for first and second points,");
            Debug.Print(" ...");
            for (i = 0; i <= nbrLineCoordinates; i++)
            {
                Debug.Print(" " + lineSymbols[i]);
            }
            Debug.Print("");
            Debug.Print("Number of arcs: " + nbrSymbols[1]);
```

```

        //Get the x,y,z coordinates of the center, start,
        //and end points of each arc, whether the arc is filled, and
        //whether a chord joining the start and end points of
        //the arc is drawn
        Debug.Print(" x,y,z coordinates of the center, start, and end points of each arc,");
        Debug.Print(" whether the arc is filled (1 = true, 0 = false),");
        Debug.Print(" and whether a chord is displayed for each arc (1 = true, 0 = false):");
        arcInfo = (double[])swEnvironment.GetSymArcs2("<GTOL-SPROF>");
        for (i = 0; i <= arcInfo.GetUpperBound(0); i++)
        {
            Debug.Print(" " + arcInfo[i]);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}

```

Analyze Text and Geometry in GTol Symbol Example (VBA)

This example shows how to analyze the text and geometry in a GTol symbol.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Gets whether <"GTOL-SPROF"> is bound.
' 3. Gets the number of lines, arcs, circles, text strings,
'    and triangles in <"GTOL-SPROF">.
' 4. Gets the number of point coordinates in the
'    lines in <"GTOL-SPROF">.
' 5. Gets the x,y,z coordinates for the first and second points
'    of each line in <"GTOL-SPROF">.
' 6. Gets the arc information for <"GTOL-SPROF">.
' 7. Examine the Immediate window.
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swEnvironment As SldWorks.Environment
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim nbrSymbols As Variant
Dim lineSymbols As Variant
Dim arcInfo As Variant
Dim nbrLineCoordinates As Long
Dim i As Long

Sub main()
    Set swApp = Application.SldWorks

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS\
2018\samples\tutorial\tolanalyst\minimum_clearance\top_plate.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)
    Set swEnvironment = swApp.GetEnvironment

    Debug.Print "GTOL library - Surface Profile symbol name:"

    'Get whether <"GTOL-SPROF"> is bound
    Dim bound As Boolean
    swEnvironment.GetIsBound "<GTOL-SPROF>", bound
    Debug.Print (" Bound? " & bound)
    Debug.Print ("")

    'Get the number of lines, arcs, circles, text strings,
    'and triangles in <"GTOL-SPROF">
    nbrSymbols = swEnvironment.GetSymEdgeCounts("<GTOL-SPROF>")
    Debug.Print (" Line count: " & nbrSymbols(0))
    Debug.Print (" Arc count: " & nbrSymbols(1))
    Debug.Print (" Circle count: " & nbrSymbols(2))
    Debug.Print (" Text count: " & nbrSymbols(3))
    Debug.Print (" Triangle count: " & nbrSymbols(4))
    Debug.Print ""
    Debug.Print ("Number of lines: " & nbrSymbols(0))

    'Get the x,y,z coordinates for the first
    'and second points of each line
    lineSymbols = swEnvironment.GetSymLines("<GTOL-SPROF>")
    nbrLineCoordinates = UBound(lineSymbols)
    Debug.Print (" Number of point coordinates: " & nbrLineCoordinates + 1)
    Debug.Print (" Point coordinates:")
    Debug.Print (" (Line 1's x,y,z coordinates for first and second points,")
    Debug.Print (" Line 2's x,y,z coordinates for first and second points,")
    Debug.Print (" ...)")
    For i = 0 To nbrLineCoordinates
        Debug.Print (" " & lineSymbols(i))
    Next i
    Debug.Print ""
    Debug.Print ("Number of arcs: " & nbrSymbols(1))

    'Get the x,y,z coordinates of the center, start, and
    'end points of each arc, whether the arc is filled, and
    'whether a chord joining the start and end points of
    'the arc is drawn
    Debug.Print " x,y,z coordinates of the center, start, and end points of each arc,"
    Debug.Print " whether the arc is filled (1 = true, 0 = false),"
    Debug.Print " and whether a chord is displayed for each arc (1 = true, 0 = false):"
    arcInfo = swEnvironment.GetSymArcs2("<GTOL-SPROF>")
    For i = 0 To UBound(arcInfo)
        Debug.Print (" " & arcInfo(i))
    Next i

```

End Sub